

Rapport Pedagogique Complet

Developpement d'un Blog Multi-Role

avec Node.js, Express et PostgreSQL

Stack Technologique

- **Backend** : Node.js + Express
- **Base de donnees** : PostgreSQL
- **Driver** : pg (sans ORM)
- **Frontend** : HTML + CSS + JavaScript Vanilla
- **Authentication** : JWT
- **Securite** : bcrypt

Licence 1 Informatique
Departement Genie Informatique

28 mai 2026

Table des matières

1	Introduction Generale	3
1.1	Qu'est-ce qu'une Application Web?	3
1.1.1	Architecture Client-Serveur	3
1.2	Qu'est-ce qu'une API?	4
1.3	Pourquoi un Blog est un Projet Pedagogique Ideal?	4
1.4	Architecture Complete du Projet	4
1.4.1	Explication du diagramme	5
1.4.2	Pourquoi une table <code>blog_tags</code> ?	5
2	Initialisation du Projet Node.js	7
2.1	Installation de Node.js	7
2.1.1	Verification de l'Installation	7
2.2	Creation du Dossier du Projet	7
2.3	Initialisation avec npm	7
2.3.1	Structure du Fichier <code>package.json</code>	8
2.3.2	Versioning Semantique (SemVer)	8
2.4	Structure Professionnelle du Projet	9
3	Installation des Dependances	11
3.1	Introduction aux Dependances	11
3.2	Express - Framework Web	11
3.3	pg - Driver PostgreSQL	12
3.4	bcrypt - Hachage de Mots de Passe	12
3.5	jsonwebtoken - Authentification JWT	13
3.6	dotenv - Gestion des Variables d'Environnement	13
3.7	cors - Partage de Ressources Cross-Origin	14
3.8	nodemon - Redemarrage Automatique	14
3.9	Configuration des Scripts	15
3.10	Erreur Courante : Module Non Trouve	16
3.11	Le Dossier <code>node_modules</code>	16
3.12	<code>package-lock.json</code>	16
4	Configuration du Serveur Express	19
4.1	Introduction a Express	19
4.2	Creation du Fichier <code>server.js</code>	19
4.3	Explication Detaillee du Code	22

4.3.1	Importation des Dependances	22
4.3.2	Initialisation de l'Application	22
4.3.3	Les Middlewares	22
4.3.4	express.json()	23
4.3.5	express.static()	23
4.3.6	Definition des Routes	23
4.3.7	Gestion des Erreurs	24
4.3.8	Demarrage du Serveur	24
4.4	Cycle de Vie d'une Requete	24
4.5	Ports HTTP	25
5	Base de Donnees PostgreSQL	27
5.1	Introduction aux Bases de Donnees Relationnelles	27
5.2	Concepts Fondamentaux	27
5.2.1	Tables	27
5.2.2	Lignes (Rows)	27
5.2.3	Colonnes (Columns)	28
5.2.4	Contraintes	28
5.2.5	Cles Primaires	28
5.2.6	Cles Etrangères	28
5.3	Pourquoi PostgreSQL ?	29
5.4	Debat : SERIAL vs UUID	29
5.4.1	SERIAL	29
5.4.2	UUID	29
5.5	Generation de UUID dans PostgreSQL	30
6	Modelisation Relationnelle	31
6.1	Introduction a la Modelisation	31
6.2	Entites du Projet Blog	31
6.3	Types de Relations	31
6.3.1	Relation Un-a-Plusieurs (One-to-Many)	31
6.3.2	Relation Plusieurs-a-Plusieurs (Many-to-Many)	32
6.4	Integrite Referentielle	32
6.5	Schema Complet de la Base de Donnees	33
6.6	Diagramme Entite-Association (ERD)	35
6.7	Explication des Contraintes	35
6.7.1	NOT NULL	35
6.7.2	UNIQUE	35
6.7.3	CHECK	35
6.7.4	DEFAULT	36
6.8	Comportement ON DELETE	36
6.8.1	CASCADE	36
6.8.2	RESTRICT	36
6.8.3	SET NULL	36

7	Connexion Node.js ↔ PostgreSQL	39
7.1	Introduction au Driver pg	39
7.2	Fichier de Configuration db.js	39
7.3	Explication du Code	41
7.3.1	Pool de Connexions	41
7.3.2	Configuration du Pool	41
7.3.3	Variables d'Environnement	42
7.4	Execution de Requetes SQL	42
7.4.1	Requete Simple	42
7.4.2	Requete avec Parametres	43
7.5	Erreurs Courantes	43
7.5.1	ECONNREFUSED	43
7.5.2	Password Authentication Failed	44
7.6	Creation de la Base de Donnees	44
7.7	Execution du Script SQL	44
7.8	Connexion à PostgreSQL depuis pgAdmin et Node	45
7.8.1	Comment savoir / réinitialiser le mot de passe utilisé par pgAdmin	45
7.8.2	Quelles informations entrer dans pgAdmin et dans '.env' pour Node	46
7.9	Authentification : Access token et Refresh token	47
8	Securite des Mots de Passe — Bcrypt	49
8.1	Introduction a la Securite des Mots de Passe	49
8.2	Pourquoi Ne Pas Stocker les Mots de Passe en Clair ?	50
8.3	Hachage vs Chiffrement	50
8.3.1	Chiffrement (Encryption)	50
8.3.2	Hachage (Hashing)	51
8.4	Le Probleme du Hachage Simple	51
8.4.1	Probleme : Rainbow Tables	51
8.5	La Solution : Le Salting	52
8.6	Bcrypt : La Solution Professionnelle	52
8.6.1	Fonctionnement de Bcrypt	53
8.6.2	Structure du Hash Bcrypt	53
8.6.3	Salt Rounds (Cout de Calcul)	53
8.7	Verification d'un Mot de Passe	54
8.8	Implementation dans l'Application	54
8.9	Bonnes Pratiques de Securite des Mots de Passe	56
9	Authentification JWT	59
9.1	Introduction a l'Authentification	59
9.2	Authentification vs Autorisation	59
9.3	Pourquoi JWT ?	59
9.3.1	Authentification Traditionnelle avec Sessions	60
9.3.2	Authentification Stateless avec JWT	60
9.4	Structure d'un JWT	60
9.4.1	Header (En-tete)	60
9.4.2	Payload (Charge Utile)	61
9.4.3	Signature	61

9.5	Implementation de la Connexion	62
9.6	Generation du Token	64
9.7	Securite de la Cle Secrete	65
9.8	Expiration du Token	65
10	Les Middlewares Express	67
10.1	Introduction aux Middlewares	67
10.2	Cycle de Vie d'une Requete	67
10.3	Types de Middlewares	67
10.3.1	Middlewares au Niveau Application	67
10.3.2	Middlewares au Niveau Routeur	68
10.3.3	Middlewares au Niveau Route	68
10.3.4	Middlewares de Gestion d'Erreurs	68
10.4	Middleware d'Authentification JWT	68
10.5	Middleware de Verification des Roles	69
10.6	Utilisation des Middlewares	70
10.7	Bonnes Pratiques de Middlewares	71
11	Routes Backend Completes	73
11.1	Introduction aux Routes	73
11.2	Routes d'Authentification	73
11.2.1	Route d'Inscription	73
11.2.2	Route de Connexion	74
11.3	Routes des Blogs	75
11.4	Routes des Commentaires	78
11.5	Validation des Donnees	80
11.6	Codes HTTP	81
12	Prevention des Injections SQL	83
12.1	Qu'est-ce qu'une Injection SQL ?	83
12.2	Prevention avec Parametres Positionnels	83
12.3	Exemples de Requetes Securisees	84
13	Gestion des Erreurs	85
13.1	Types d'Erreurs	85
13.1.1	Erreurs Client (4xx)	85
13.1.2	Erreurs Serveur (5xx)	85
13.2	Try-Catch	85
13.3	Middleware d'Erreur Global	86
14	Frontend HTML/CSS/JS	87
14.1	Introduction au Frontend	87
14.2	Page d'Accueil (index.html)	87
14.3	Page de Connexion (login.html)	88
14.4	JavaScript Frontend (app.js)	88
14.5	JavaScript de Connexion (login.js)	90
14.6	CSS (style.css)	91

15	Debugage et Resolution de Problemes	93
15.1	Introduction au Debugage	93
15.2	Outils de Debugage	93
15.2.1	console.log()	93
15.2.2	Node.js Inspector	93
15.2.3	VS Code Debugger	94
15.3	Erreurs Courantes Node.js	94
15.3.1	Module Not Found	94
15.3.2	EADDRINUSE	95
15.4	Erreurs Courantes PostgreSQL	95
15.4.1	Connection Refused	95
15.4.2	Password Authentication Failed	95
15.5	Strategie de Debugage	95
16	Deploiement et Production	97
16.1	Introduction au Deploiement	97
16.2	Preparation pour la Production	97
16.2.1	Variables d'Environnement	97
16.2.2	NODE_ENV	97
16.3	PM2 - Process Manager	98
16.3.1	Installation de PM2	98
16.3.2	Demarrage avec PM2	98
16.3.3	Commandes PM2	98
16.4	Nginx - Reverse Proxy	98
16.4.1	Configuration Nginx	98
16.5	HTTPS avec Let's Encrypt	99
16.6	Base de Donnees en Production	99
17	Bonnes Pratiques de Developpement	101
17.1	Clean Code	101
17.1.1	Noms Descriptifs	101
17.1.2	Fonctions Courtes	101
17.2	Principe DRY	102
17.3	Principes SOLID	102
17.4	Validation des Donnees	103
17.5	Gestion des Secrets	103
17.6	Documentation	103
18	Extensions et Ameliorations Possibles	105
18.1	Introduction	105
18.2	Systeme de Likes	105
18.3	Tags et Categories	105
18.4	Recherche Full-Text	105
18.5	Pagination	106
18.6	Upload d'Images	106
18.7	Notifications par Email	106
18.8	Refresh Tokens	106

18.8.1 Pourquoi utiliser un refresh token ?	107
18.8.2 Flux typique	107
18.9 Rate Limiting	107
18.10 WebSocket pour Commentaires en Temps Reel	108
18.11 Caching avec Redis	108
19 Conclusion	109
19.1 Recapitulatif du Projet	109
19.2 Competences Acquises	109
19.2.1 Competences Techniques	109
19.2.2 Competences en Securite	109
19.2.3 Competences en Architecture	110
19.3 Perspectives d'Avenir	110
19.3.1 Poursuite de l'Apprentissage	110
19.3.2 Ameliorations du Projet	110
19.4 Conclusion Finale	110
Bibliographie	111

Avant-propos

Ce document a été rédigé par Joachim BANGIRAHE en tant qu'encadreur et formateur. Je m'adresse directement à vous, **étudiants**, pour vous guider pas à pas dans la conception et l'implémentation d'un blog multi-role.

Mon objectif est de vous proposer des explications claires, des exemples concrets et des bonnes pratiques que vous pourrez reprendre dans vos projets et vos TP. N'hésitez pas à tester les exemples fournis et à expérimenter les variantes.

Bonne lecture et bon travail,

Joachim BANGIRAHE – Encadreur

Chapitre 1

Introduction Generale

1.1 Qu'est-ce qu'une Application Web ?

Une application web est un logiciel qui fonctionne dans un navigateur web et communique avec un serveur via le protocole HTTP. Contrairement aux applications desktop traditionnelles qui sont installées localement sur un ordinateur, les applications web sont accessibles depuis n'importe quel appareil connecté à Internet.

Definition

Application Web : Programme informatique accessible via un navigateur web, utilisant les technologies web standard (HTML, CSS, JavaScript) et communiquant avec un serveur distant pour stocker et traiter les données.

1.1.1 Architecture Client-Serveur

L'architecture web moderne repose sur la séparation entre le **client** (frontend) et le **serveur** (backend) :

- **Frontend (Client)** : L'interface utilisateur que l'on voit dans le navigateur. Il est responsable de l'affichage, de l'interaction avec l'utilisateur et de l'envoi des requêtes au serveur.
- **Backend (Serveur)** : Le "cerveau" de l'application qui traite les requêtes, exécute la logique métier et communique avec la base de données.
- **Base de données** : Le système de stockage persistant qui conserve les données de l'application (utilisateurs, articles, commentaires, etc.).

 Analogie**Analogie du Restaurant :**

- Le **client** est le client du restaurant qui consulte le menu et passe commande.
- Le **backend** est le chef de cuisine qui recoit la commande, prepare le plat selon les recettes.
- La **base de donnees** est le garde-manger qui stocke les ingredients et les recettes.

1.2 Qu'est-ce qu'une API?

 Definition

API (Application Programming Interface) : Ensemble de regles et de protocoles qui permettent a differentes applications logicielles de communiquer entre elles. Dans le contexte web, une API REST definit comment le client peut demander des donnees au serveur et comment le serveur doit repondre.

Une API REST (Representational State Transfer) utilise les methodes HTTP standard :

- **GET** : Recuperer des donnees (lecture)
- **POST** : Creer de nouvelles donnees (creation)
- **PUT/PATCH** : Modifier des donnees existantes (mise a jour)
- **DELETE** : Supprimer des donnees (suppression)

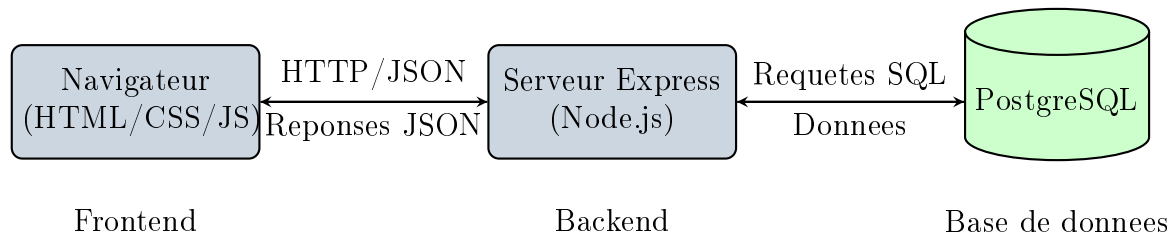
1.3 Pourquoi un Blog est un Projet Pedagogique Ideal ?

Le projet de blog est particulierement adapte a l'apprentissage du developpement backend car :

- Il implique des **relations complexes** entre les donnees (utilisateurs, articles, commentaires, tags).
- Il necessite une **authentification** et une **autorisation** (roles utilisateur).
- Il illustre parfaitement le **CRUD** (Create, Read, Update, Delete).
- Il permet de comprendre la **securite** (injection SQL, hachage de mots de passe).
- Il est suffisamment simple pour etre compris rapidement, mais assez complexe pour etre pedagogique.

1.4 Architecture Complete du Projet

Voici l'architecture complete de notre application :



1.4.1 Explication du diagramme

Le diagramme Entite-Association (ERD) ci-dessus montre les entites principales et leurs relations :

- **USERS** : chaque utilisateur a un identifiant unique et peut etre auteur de plusieurs BLOGS et COMMENTAIRES.
- **BLOGS** : represente un article. Un blog appartient a un seul auteur (relation 1 :N depuis USERS).
- **COMMENTAIRES** : chaque commentaire est rattache a un blog et a un auteur.
- **TAGS** : mots-clefs reutilisables pour qualifier un blog (ex : "nodejs", "postgres").
- **BLOG**
TAGS : table de jonction (relation N : M) qui relie BLOGS et TAGS.

1.4.2 Pourquoi une table blog_tags ?

Dans une relation many-to-many, on ne peut pas représenter directement plusieurs tags pour un blog dans une colonne ordinaire sans duplication ou violation de la normalisation. La table `blog_tags` :

- Permet d'associer un nombre arbitraire de tags a un blog sans dupliquer les tags eux-memes.
- Facilite les recherches : on peut rechercher tous les blogs ayant un tag donne sans analyser des listes serialisees.
- Rend la maintenance plus simple (suppression en cascade, contraintes de cle etrangere).

Listing 1.1 – Exemple - lier un blog a deux tags

```

1 -- Supposons qu'on ait un blog_id et deux tag_id
2 INSERT INTO blog_tags (blog_id, tag_id) VALUES ('blog-uuid-1', 'tag-uuid
  -nodejs');
3 INSERT INTO blog_tags (blog_id, tag_id) VALUES ('blog-uuid-1', 'tag-uuid
  -postgres');
```

Listing 1.2 – Exemple - recuperer les tags d'un blog

```

1 SELECT t.nom
2 FROM tags t
3 JOIN blog_tags bt ON t.id = bt.tag_id
4 WHERE bt.blog_id = 'blog-uuid-1';
```

 Architecture**Flux de Requete Complet :**

1. L'utilisateur navigue sur le site dans son navigateur.
2. Le navigateur envoie une requete HTTP au serveur Express.
3. Express traite la requete via les middlewares et les routes.
4. Le serveur interroge PostgreSQL via le driver pg.
5. PostgreSQL retourne les donnees au serveur.
6. Le serveur formate la reponse en JSON et l'envoie au navigateur.
7. Le navigateur affiche les donnees a l'utilisateur.

Chapitre 2

Initialisation du Projet Node.js

2.1 Installation de Node.js

Node.js est un environnement d'exécution JavaScript cote serveur. Il permet d'exécuter du JavaScript en dehors du navigateur, ce qui est essentiel pour le développement backend.

Definition

Node.js : Environnement d'exécution JavaScript open-source, multiplateforme, base sur le moteur V8 de Chrome, permettant d'exécuter du JavaScript cote serveur.

2.1.1 Verification de l'Installation

Avant de commencer, vérifiez que Node.js est installé sur votre machine :

Listing 2.1 – Verification de l'installation de Node.js

```
1 node --version
2 npm --version
```

Si Node.js n'est pas installé, téléchargez-le depuis <https://nodejs.org/> et installez la version LTS (Long Term Support) recommandée.

2.2 Creation du Dossier du Projet

Créez un nouveau dossier pour votre projet :

Listing 2.2 – Creation du dossier du projet

```
1 mkdir blog-multi-role
2 cd blog-multi-role
```

2.3 Initialisation avec npm

npm (Node Package Manager) est le gestionnaire de paquets de Node.js. Il permet de gérer les dépendances du projet.

Listing 2.3 – Initialisation du projet avec npm

```
1 npm init -y
```

L'option `-y` permet de generer un fichier `package.json` avec les valeurs par défaut sans poser de questions interactives.

2.3.1 Structure du Fichier `package.json`

Le fichier `package.json` est le "carte d'identité" de votre projet. Il contient des metadonnées essentielles :

```
1 {  
2   "name": "blog-multi-role",  
3   "version": "1.0.0",  
4   "description": "Blog multi-role avec Node.js, Express et PostgreSQL",  
5   "main": "server.js",  
6   "scripts": {  
7     "start": "node server.js",  
8     "dev": "nodemon server.js"  
9   },  
10  "keywords": ["blog", "nodejs", "express", "postgresql"],  
11  "author": "Licence 1 Informatique",  
12  "license": "ISC",  
13  "dependencies": {  
14    "express": "^4.18.2",  
15    "pg": "^8.11.0",  
16    "bcrypt": "^5.1.0",  
17    "jsonwebtoken": "^9.0.0",  
18    "dotenv": "^16.0.3"  
19  },  
20  "devDependencies": {  
21    "nodemon": "^2.0.22"  
22  }  
23 }
```

Listing 2.4 – Exemple de `package.json`

Definition

Explication des champs :

- **name** : Nom unique du projet
- **version** : Version du projet (suivant le semantique versioning)
- **main** : Point d'entree principal de l'application
- **scripts** : Commandes personnalisées pour executer le projet
- **dependencies** : Bibliothèques nécessaires en production
- **devDependencies** : Bibliothèques nécessaires uniquement en developpement

2.3.2 Versioning Semantique (SemVer)

Le versioning semantique suit le format `MAJEUR.MINEUR.CORRECTIF` :

- **MAJEUR** : Changements incompatibles avec l'API precedente (ex : 1.0.0 → 2.0.0)
- **MINEUR** : Nouvelles fonctionnalites compatibles (ex : 1.0.0 → 1.1.0)
- **CORRECTIF** : Corrections de bugs compatibles (ex : 1.0.0 → 1.0.1)

2.4 Structure Professionnelle du Projet

Une structure de projet bien organisee est essentielle pour la maintenabilite et la collaboration.

Listing 2.5 – Structure complete du projet

```

1 \begin{lstlisting}[style=bash, caption=Structure complete du projet]
2 blog-multi-role/
3 |-- server.js           # Point d'entree du serveur
4 |-- db.js              # Configuration de la base de donnees
5 |-- .env               # Variables d'environnement (NE PAS COMMIT)
6 |-- .gitignore         # Fichiers a ignorer par Git
7 |-- package.json       # Configuration du projet
8 |-- package-lock.json  # Verrouillage des versions
9 |-- middleware/        # Middlewares Express
10    |-- auth.js        # Verification JWT
11    |-- roles.js       # Verification des roles
12 |-- routes/          # Routes de l'API
13    |-- auth.js        # Routes d'authentification
14    |-- blogs.js       # Routes des blogs
15    |-- commentaires.js # Routes des commentaires
16 |-- public/          # Fichiers statiques
17    |-- index.html     # Page d'accueil
18    |-- login.html     # Page de connexion
19    |-- register.html  # Page d'inscription
20    |-- css/
21        |-- style.css  # Styles CSS
22    |-- js/
23        |-- app.js     # JavaScript frontend
24 |-- sql/              # Scripts SQL
25    |-- schema.sql     # Schema de la base de donnees
26 |-- README.md        # Documentation du projet

```

Architecture

Explication des Dossiers :

- **middleware/** : Contient les fonctions intermediaires qui traitent les requetes avant les routes (authentification, validation, etc.).
- **routes/** : Contient les definitions des routes de l'API, organisees par fonctionnalite.
- **public/** : Contient les fichiers statiques servis aux clients (HTML, CSS, JavaScript, images).
- **sql/** : Contient les scripts SQL pour la creation et la manipulation de la base de donnees.

 Bonne Pratique

Principe de Separation des Preoccupations : Chaque dossier a une responsabilite unique et bien definie. Cette separation rend le code plus maintenable, testable et comprehensible.

Chapitre 3

Installation des Dependances

3.1 Introduction aux Dependances

Les dependances sont des bibliotheques externes que votre projet utilise pour fonctionner. Elles sont declarees dans `package.json` et installees dans le dossier `node_modules/`.

Definition

Dependance : Bibliotheque ou module externe dont votre projet depend pour fonctionner. Elle peut etre une dependance de production (necessaire en production) ou de developpement (necessaire uniquement pendant le developpement).

3.2 Express - Framework Web

Listing 3.1 – Installation d'Express

```
1 npm install express
```

Definition

Express.js : Framework web minimal et flexible pour Node.js, fournissant un ensemble robuste de fonctionnalites pour les applications web et mobiles. Il simplifie la creation de serveurs HTTP et la gestion des routes.

Analogie

Analogie : Express est comme un "kit de construction" pour serveurs web. Au lieu de devoir construire chaque brique (gestion des routes, parsing du corps des requetes, gestion des cookies) vous-meme, Express vous fournit ces briques prefabriquees.

Pourquoi Express ?

- Simplifie la creation de serveurs HTTP
- Gere automatiquement le routing (les routes)

- Parse automatiquement les requetes JSON
- Supporte les middlewares pour la modularite
- Tres populaire et bien documente

3.3 pg - Driver PostgreSQL

Listing 3.2 – Installation du driver pg

```
1 npm install pg
```

Definition

pg : Driver PostgreSQL non-bloquant pour Node.js. Il permet d'exécuter des requetes SQL depuis Node.js et de communiquer avec une base de donnees PostgreSQL.

Analogie

Analogie : Le driver pg est comme un "traducteur" entre Node.js (qui parle JavaScript) et PostgreSQL (qui parle SQL). Il convertit les requetes JavaScript en requetes SQL que PostgreSQL peut comprendre.

Pourquoi pg et pas un ORM ?

- **Apprentissage** : Comprendre le SQL brut est essentiel pour un ingenieur backend
- **Performance** : Les drivers sont generalement plus performants que les ORM
- **Controle** : Vous avez un controle total sur les requetes SQL
- **Transparence** : Vous voyez exactement ce qui est envoye a la base de donnees

3.4 bcrypt - Hachage de Mots de Passe

Listing 3.3 – Installation de bcrypt

```
1 npm install bcrypt
```

Definition

bcrypt : Bibliotheque de hachage de mots de passe concue pour etre lente et resistente aux attaques par force brute. Elle utilise un sel aleatoire pour chaque mot de passe.

💡 Analogie

Analogie : Le hachage est comme une "empreinte digitale" irreversible. Vous pouvez creer une empreinte a partir d'un doigt, mais vous ne pouvez pas recreer le doigt a partir de l'empreinte. Le sel est comme ajouter du "poivre" unique a chaque empreinte pour qu'elles soient toutes differentes meme pour le meme mot de passe.

Pourquoi bcrypt ?

- **Securite** : Concu specifiquement pour les mots de passe
- **Sel automatique** : Genere un sel unique pour chaque mot de passe
- **Lenteur** : Lent par design pour ralentir les attaques par force brute
- **Standard** : Standard de l'industrie pour le hachage de mots de passe

3.5 jsonwebtoken - Authentication JWT

Listing 3.4 – Installation de jsonwebtoken

```
1 npm install jsonwebtoken
```

📖 Definition

JWT (JSON Web Token) : Standard ouvert (RFC 7519) pour creer des tokens d'accès securises. Un JWT est un token compact et autonome qui contient des informations (claims) et est signe cryptographiquement.

💡 Analogie

Analogie du Badge : Un JWT est comme un badge d'étudiant securise. Il contient votre nom, votre role (etudiant/professeur), et une date d'expiration. Le badge est signe par l'universite, donc impossible a falsifier. Vous le presentez pour acceder aux batiments, et le gardien verifie la signature.

Pourquoi JWT ?

- **Stateless** : Le serveur n'a pas besoin de stocker les sessions
- **Compact** : Peut etre envoye dans l'en-tete HTTP
- **Securise** : Signe cryptographiquement
- **Standard** : Standard de l'industrie pour l'authentification moderne

3.6 dotenv - Gestion des Variables d'Environnement

Listing 3.5 – Installation de dotenv

```
1 npm install dotenv
```

Definition

dotenv : Module qui charge les variables d'environnement depuis un fichier `.env` dans `process.env`. Cela permet de separer la configuration du code.

Analogie

Analogie : Le fichier `.env` est comme un "fichier de configuration secret" qui contient toutes les informations sensibles (mots de passe, clés API) separement du code. C'est comme avoir un coffre-fort separe pour vos secrets.

Pourquoi dotenv ?

- **Securite** : Evite de hardcoder les secrets dans le code
- **Flexibilite** : Facile de changer de configuration selon l'environnement (dev/prod)
- **Standard** : Standard de l'industrie pour la gestion des secrets

3.7 cors - Partage de Ressources Cross-Origin

Listing 3.6 – Installation de cors

```
1 npm install cors
```

Definition

CORS (Cross-Origin Resource Sharing) : Mecanisme de securite base sur des en-tetes HTTP qui permet a un serveur d'indiquer quelles origines (domaines) sont autorisees a acceder a ses ressources.

Analogie

Analogie : CORS est comme un "systeme de controle d'accès" pour les requetes web. Si votre site (example.com) veut acceder a une API sur (api.example.com), CORS verifie que c'est autorise. C'est comme un gardien qui verifie les badges avant de laisser entrer.

Pourquoi cors ?

- **Securite** : Protege contre les requetes malveillantes cross-origin
- **Simplicité** : Simplifie la configuration CORS dans Express
- **Necessaire** : Obligatoire pour les applications frontend separees

3.8 nodemon - Redemarrage Automatique

Listing 3.7 – Installation de nodemon (dependance de developpement)

```
1 npm install --save-dev nodemon
```

Definition

nodemon : Outil qui surveille les modifications de fichiers dans votre projet et redemarre automatiquement le serveur Node.js lors des changements. Indispensable pour le developpement.

Analogie

Analogie : nodemon est comme un "assistant vigilant" qui surveille votre travail. Des que vous sauvegardez un fichier, il redemarre automatiquement le serveur pour que vous puissiez voir les changements immediatement. Sans lui, vous devriez arreter et redemarrer le serveur manuellement a chaque modification.

Pourquoi nodemon ?

- **Productivite** : Evite les redemarrages manuels frequents
- **Developpement uniquement** : Pas necessaire en production
- **Standard** : Outil standard dans l'ecosysteme Node.js

3.9 Configuration des Scripts

Modifiez la section `scripts` dans `package.json` :

```
1 {  
2   "scripts": {  
3     "start": "node server.js",  
4     "dev": "nodemon server.js"  
5   }  
6 }
```

Listing 3.8 – Configuration des scripts dans package.json

Bonne Pratique

Scripts Standard :

- **npm start** : Commande pour demarrer en production
- **npm run dev** : Commande pour le developpement avec nodemon

3.10 Erreur Courante : Module Non Trouve

! Erreur

Erreur : Cannot find module 'express'

Cause : La dependance n'a pas ete installee ou le dossier `node_modules` est manquant.

Diagnostic :

- Verifiez que vous avez execute `npm install`
- Verifiez que le dossier `node_modules` existe
- Verifiez que `package.json` contient la dependance

Resolution :

```
1 npm install
```

Explication : La commande `npm install` lit `package.json` et installe toutes les dependances listees dans le dossier `node_modules`.

3.11 Le Dossier `node_modules`

Le dossier `node_modules` contient toutes les dependances installees et leurs dependances transitives.

! Attention

Important : Le dossier `node_modules` ne doit **jamais** etre commite dans Git. Il est trop volumineux et peut etre regenere avec `npm install`.

Listing 3.9 – Contenu de `.gitignore`

```
1 node_modules/  
2 .env  
3 .DS_Store  
4 *.log
```

📖 Definition

Dependances Transitives : Les dependances des dependances. Par exemple, si Express depend de 5 autres bibliotheques, ces 5 bibliotheques seront egalement installees dans `node_modules`.

3.12 `package-lock.json`

Le fichier `package-lock.json` est genere automatiquement par npm. Il verrouille les versions exactes de toutes les dependances installees.

 Definition

package-lock.json : Fichier qui enregistre la version exacte de chaque dépendance installée, y compris les dépendances transitives. Il assure que tous les développeurs installent exactement les mêmes versions.

 Bonne Pratique**Bonnes Pratiques :**

- Committez toujours `package-lock.json`
- Ne modifiez jamais `package-lock.json` manuellement
- Utilisez `npm ci` en production pour une installation reproductible

Chapitre 4

Configuration du Serveur Express

4.1 Introduction a Express

Express est un framework web minimal pour Node.js qui fournit des fonctionnalites robustes pour les applications web et mobiles. Il simplifie la creation de serveurs HTTP et la gestion des routes.

Definition

Framework Web : Ensemble de bibliotheques et d'outils qui fournissent une structure standard pour developper des applications web. Un framework definit des conventions et fournit des fonctionnalites pretes a l'emploi.

4.2 Creation du Fichier server.js

Creez le fichier `server.js` a la racine du projet. Ce sera le point d'entree de votre application.

```
1 // Importation des dependances
2 const express = require('express');
3 const cors = require('cors');
4 const path = require('path');
5 require('dotenv').config();
6
7 // Importation des routes
8 const authRoutes = require('./routes/auth');
9 const blogRoutes = require('./routes/blogs');
10 const commentaireRoutes = require('./routes/commentaires');
11
12 // Importation de la configuration de la base de donnees
13 const db = require('./db');
14
15 // Initialisation de l'application Express
16 const app = express();
17 const PORT = process.env.PORT || 3000;
18
```

```
19 //  
    =====  
20 // MIDDLEWARES GLOBAUX  
21 //  
    =====  
22  
23 // Middleware CORS - Permet les requetes cross-origin  
24 app.use(cors());  
25  
26 // Middleware pour parser le JSON dans le corps des requetes  
27 app.use(express.json());  
28  
29 // Middleware pour parser les donnees URL-encoded  
30 app.use(express.urlencoded({ extended: true }));  
31  
32 // Middleware pour servir les fichiers statiques  
33 app.use(express.static(path.join(__dirname, 'public')));  
34  
35 //  
    =====  
36 // MIDDLEWARE DE LOGGING  
37 //  
    =====  
38  
39 // Middleware pour logger chaque requete  
40 app.use((req, res, next) => {  
41     console.log('[${new Date().toISOString()}] ${req.method} ${req.path  
42         }');  
43     next();  
44 });  
45 //  
    =====  
46 // ROUTES  
47 //  
    =====  
48  
49 // Routes d'authentification  
50 app.use('/api/auth', authRoutes);  
51  
52 // Routes des blogs  
53 app.use('/api/blogs', blogRoutes);  
54  
55 // Routes des commentaires  
56 app.use('/api/commentaires', commentaireRoutes);  
57  
58 // Route de test  
59 app.get('/api/test', (req, res) => {  
60     res.json({ message: 'Le serveur fonctionne correctement !' });  
61 });
```

```
62 //
63 // =====
64 // GESTION DES ERREURS 404
65 // =====
66
67 // Middleware pour les routes non trouvees
68 app.use((req, res) => {
69     res.status(404).json({ error: 'Route non trouvee' });
70 });
71
72 // =====
73 // GESTION DES ERREURS GLOBALES
74 // =====
75
76 // Middleware de gestion des erreurs
77 app.use((err, req, res, next) => {
78     console.error('Erreur:', err.stack);
79     res.status(500).json({ error: 'Erreur interne du serveur' });
80 });
81
82 // =====
83 // DEMARRAGE DU SERVEUR
84 // =====
85
86 // Demarrage du serveur
87 app.listen(PORT, () => {
88     console.log('\n=====');
89     console.log('Serveur demarre sur le port ${PORT}');
90     console.log('URL : http://localhost:${PORT}');
91     console.log('=====\n');
92 });
93
94 // Gestion de l'arret gracieux
95 process.on('SIGINT', () => {
96     console.log('\nArret du serveur...');
97     db.end(() => {
98         console.log('Connexion a la base de donnees fermee');
99         process.exit(0);
100     });
101 });
```

Listing 4.1 – server.js - Configuration complete du serveur Express

4.3 Explication Detaillee du Code

4.3.1 Importation des Dependances

```
1 const express = require('express');
```

Definition

require() : Fonction Node.js pour importer des modules. Dans CommonJS (le systeme de modules de Node.js), **require** charge un module et retourne son export.

Analogie

Analogie : **require** est comme "emprunter un livre" dans une bibliotheque. Vous demandez un livre (module) specifique, et la bibliotheque vous le donne pour que vous puissiez l'utiliser.

4.3.2 Initialisation de l'Application

```
1 const app = express();
```

Definition

express() : Fonction qui cree une application Express. Cette application est un objet qui represente votre serveur web et possede des methodes pour configurer les routes, les middlewares, etc.

4.3.3 Les Middlewares

```
1 app.use(express.json());
```

Definition

Middleware : Fonction qui a acces a l'objet requete (req), l'objet reponse (res), et la fonction middleware suivante (next) dans le cycle requete-reponse de l'application.

Analogie

Analogie du Concierge : Un middleware est comme un concierge dans un immeuble. Quand quelqu'un arrive (requete), le concierge verifie son identite, enregistre son passage, puis lui ouvre la porte (next) pour qu'il puisse continuer vers sa destination.

4.3.4 `express.json()`

```
1 app.use(express.json());
```

Definition

express.json() : Middleware qui parse le corps des requetes HTTP en JSON. Il convertit automatiquement les donnees JSON envoyees par le client en objet JavaScript accessible via `req.body`.

Pourquoi est-ce necessaire ? Sans ce middleware, `req.body` serait `undefined` et vous ne pourriez pas acceder aux donnees envoyees par le client.

4.3.5 `express.static()`

```
1 app.use(express.static(path.join(__dirname, 'public')));
```

Definition

express.static() : Middleware qui sert des fichiers statiques (HTML, CSS, JavaScript, images). Il rend les fichiers du dossier specifie accessibles via HTTP.

Analogie

Analogie : `express.static` est comme un "distributeur automatique" de fichiers. Quand un client demande un fichier (ex : `/style.css`), le middleware trouve le fichier correspondant dans le dossier `public` et l'envoie au client.

4.3.6 Definition des Routes

```
1 app.use('/api/auth', authRoutes);
```

Definition

Route : Chemin URL qui correspond a une fonction de gestionnaire specifique. Quand une requete correspond au chemin et a la methode HTTP, le gestionnaire associe est execute.

Analogie

Analogie : Une route est comme une "adresse" dans un systeme de routage postal. Quand vous envoyez une lettre a une adresse specifique, le systeme de routage sait exactement ou la livrer.

4.3.7 Gestion des Erreurs

```
1 app.use((err, req, res, next) => {  
2   console.error('Erreur:', err.stack);  
3   res.status(500).json({ error: 'Erreur interne du serveur' });  
4 });
```

Definition

Middleware d'Erreur : Middleware special avec quatre arguments (err, req, res, next). Express le reconnait automatiquement comme middleware de gestion d'erreurs.

Bonne Pratique

Bonnes Pratiques de Gestion d'Erreurs :

- Toujours logger les erreurs cote serveur
- Ne jamais exposer les details de l'erreur au client en production
- Utiliser des codes HTTP appropries (400, 401, 403, 404, 500)
- Fournir des messages d'erreur clairs mais non techniques

4.3.8 Demarrage du Serveur

```
1 app.listen(PORT, () => {  
2   console.log('Serveur démarre sur le port ${PORT}');  
3 });
```

Definition

app.listen() : Methode qui lie et ecoute les connexions sur le port specifie. Le serveur commence a accepter les requetes HTTP sur ce port.

Analogie

Analogie : **app.listen()** est comme "ouvrir un bureau" et afficher un numero de telephone. Une fois ouvert, les clients peuvent appeler (faire des requetes) a ce numero (port).

4.4 Cycle de Vie d'une Requete

Voici le cycle complet d'une requete HTTP dans Express :

1. Le client envoie une requete HTTP au serveur
2. Express recoit la requete

3. Les middlewares globaux s'exécutent dans l'ordre (CORS, JSON parsing, logging)
4. Express trouve la route correspondante
5. Les middlewares spécifiques à la route s'exécutent
6. Le gestionnaire de route s'exécute
7. La réponse est envoyée au client
8. Les middlewares d'erreur s'exécutent si une erreur survient



🔧 Architecture

Chaine de Middlewares : Les middlewares s'exécutent séquentiellement dans l'ordre où ils sont définis. Chaque middleware peut soit terminer le cycle (envoyer une réponse), soit passer le contrôle au middleware suivant avec `next()`.

4.5 Ports HTTP

📖 Definition

Port : Numérique identifiant un processus spécifique sur un ordinateur. Les ports permettent à plusieurs applications de communiquer simultanément sur le même ordinateur.

Ports Courants :

- **80** : HTTP (non sécurisé)
- **443** : HTTPS (sécurisé)
- **3000** : Port de développement courant pour Node.js
- **5000** : Port de développement alternatif
- **8080** : Port de développement alternatif

✅ Bonne Pratique

Bonnes Pratiques :

- Utiliser des ports supérieurs à 1024 pour le développement (ports < 1024 nécessitent des privilèges administrateur)
- Utiliser des variables d'environnement pour le port
- Ne jamais hardcoder le port en production

Chapitre 5

Base de Donnees PostgreSQL

5.1 Introduction aux Bases de Donnees Relationnelles

Definition

Base de Donnees Relationnelle : Systeme de gestion de base de donnees (SGBD) qui organise les donnees en tables avec des relations entre elles. Les donnees sont structurees selon le modele relationnel invente par E.F. Codd en 1970.

Analogie

Analogie du Classeur : Une base de donnees relationnelle est comme un classeur geant avec des feuilles (tables). Chaque feuille a des colonnes (champs) et des lignes (enregistrements). Les feuilles sont reliees entre elles par des references croisees (cles etrangeres).

5.2 Concepts Fondamentaux

5.2.1 Tables

Definition

Table : Structure qui organise les donnees en lignes et colonnes. Chaque table represente une entite (ex : utilisateurs, articles, commentaires).

5.2.2 Lignes (Rows)

Definition

Ligne (Row) : Enregistrement individuel dans une table. Chaque ligne represente une instance d'une entite (ex : un utilisateur specifique, un article specifique).

5.2.3 Colonnes (Columns)

Definition

Colonne (Column) : Champ qui contient un type de donnees specifique pour chaque ligne (ex : nom, email, date de creation).

5.2.4 Contraintes

Definition

Contrainte : Regle appliquee a une colonne ou une table pour garantir l'integrite des donnees (ex : NOT NULL, UNIQUE, CHECK).

5.2.5 Cles Primaires

Definition

Cle Primaire (Primary Key) : Colonne (ou ensemble de colonnes) qui identifie de maniere unique chaque ligne d'une table. Chaque table doit avoir exactement une cle primaire.

Analogie

Analogie du Numero de Securite Sociale : La cle primaire est comme le numero de securite sociale. Chaque personne a un numero unique qui l'identifie. Il n'y a pas deux personnes avec le meme numero.

5.2.6 Cles Etrangeres

Definition

Cle Etrangere (Foreign Key) : Colonne qui fait reference a la cle primaire d'une autre table. Elle etablit une relation entre deux tables.

Analogie

Analogie de la Reference : Une cle etrangere est comme une "reference" dans une lettre. Si vous ecrivez une lettre a quelqu'un et mentionnez "Voir le dossier #123", le #123 est une reference a un autre dossier.

5.3 Pourquoi PostgreSQL ?

PostgreSQL est un systeme de gestion de base de donnees relationnelle objet, open-source et extremement robuste.

- **Open Source** : Gratuit et communautaire
- **Robuste** : Tres fiable et stable
- **Conforme aux Standards** : Respecte les standards SQL
- **Extensible** : Supporte les extensions et les types personnalisés
- **Performant** : Excellent pour les applications complexes
- **Securise** : Supporte l'authentification, le chiffrement, les roles

5.4 Debat : SERIAL vs UUID

5.4.1 SERIAL

Definition

SERIAL : Type de donnees PostgreSQL qui genere automatiquement des entiers sequentiels (1, 2, 3, ...). C'est l'equivalent de AUTO_INCREMENT dans MySQL.

Avantages de SERIAL :

- Simple a utiliser
- Compact (4 octets)
- Naturellement ordonne
- Facile a deboguer

Inconvenients de SERIAL :

- **Previsible** : Les attaquants peuvent deviner les IDs
- **Enumerable** : Possible de parcourir toutes les ressources
- **Probleme d'evolutivite** : Dans les systemes distribues, difficile de garantir l'unicite
- **Fuite d'information** : Le nombre d'utilisateurs est visible

5.4.2 UUID

Definition

UUID (Universally Unique Identifier) : Identifiant unique de 128 bits, generalement represente par 36 caracteres hexadecimaux (ex : 550e8400-e29b-41d4-a716-446655440000).

Avantages de UUID :

- **Imprevisible** : Impossible de deviner les IDs
- **Unique globalement** : Garantie d'unicite meme dans les systemes distribues
- **Securise** : Ne revele pas le nombre d'enregistrements
- **URL-safe** : Peut etre utilise directement dans les URLs
- **Pas de collision** : Probabilite de collision negligeable
- Inconvenients de UUID** :
 - Plus volumineux (16 octets vs 4 octets)
 - Non ordonne (peut fragmenter les index)
 - Moins lisible pour les humains

Securite

Recommandation de Securite : Pour ce projet pedagogique, nous utilisons UUID pour les raisons suivantes :

- **Securite** : Empeche l'enumeration des ressources
- **Pedagogique** : Montre les meilleures pratiques de securite
- **Moderne** : Approche recommandee pour les nouvelles applications

5.5 Generation de UUID dans PostgreSQL

PostgreSQL fournit l'extension `pgcrypto` pour generer des UUID cryptographiquement securises.

Code : Activation de l'extension `pgcrypto`

```
1 CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Definition

pgcrypto : Extension PostgreSQL qui fournit des fonctions cryptographiques, y compris la generation de UUID aleatoires avec `gen_random_uuid()`.

Listing 5.1 – Generation d'un UUID

```
1 SELECT gen_random_uuid();
2 -- Resultat : 550e8400-e29b-41d4-a716-446655440000
```

Bonne Pratique

Bonnes Pratiques UUID :

- Utiliser `gen_random_uuid()` plutot que `uuid_generate_v4()` (plus cryptographiquement securise)
- Stocker les UUID dans des colonnes de type `UUID`
- Ne pas generer les UUID dans l'application (generation cote base de donnees)

Chapitre 6

Modelisation Relationnelle

6.1 Introduction a la Modelisation

La modelisation relationnelle est le processus de conception de la structure d'une base de donnees relationnelle. Elle definit les tables, les colonnes, les contraintes et les relations entre les tables.

Definition

Modelisation Relationnelle : Processus de conception qui transforme les exigences du monde reel en une structure de base de donnees relationnelle, en identifiant les entites, leurs attributs et leurs relations.

6.2 Entites du Projet Blog

Notre blog multi-role comporte les entites suivantes :

- **Utilisateurs** : Personnes qui s'inscrivent et se connectent
- **Blogs** : Articles publies par les utilisateurs
- **Commentaires** : Reponses aux blogs
- **Tags** : Categories ou etiquettes pour les blogs

6.3 Types de Relations

6.3.1 Relation Un-a-Plusieurs (One-to-Many)

Definition

Relation One-to-Many : Relation ou un enregistrement d'une table peut etre associe a plusieurs enregistrements d'une autre table, mais un enregistrement de la deuxieme table n'est associe qu'a un seul enregistrement de la premiere.

 Analogie

Analogie Parent-Enfant : Un parent peut avoir plusieurs enfants, mais chaque enfant n'a qu'un seul parent biologique.

Exemples dans notre projet :

- Un utilisateur peut écrire plusieurs blogs (relation Users $\rightarrow$ Blogs)
- Un blog peut avoir plusieurs commentaires (relation Blogs $\rightarrow$ Commentaires)
- Un utilisateur peut écrire plusieurs commentaires (relation Users $\rightarrow$ Commentaires)

6.3.2 Relation Plusieurs-a-Plusieurs (Many-to-Many)

 Definition

Relation Many-to-Many : Relation où plusieurs enregistrements d'une table peuvent être associés à plusieurs enregistrements d'une autre table. Cette relation nécessite une table de jonction (table associative).

 Analogie

Analogie Réseau Social : Dans un réseau social, une personne peut avoir plusieurs amis, et chaque ami peut avoir plusieurs amis. C'est une relation many-to-many.

Exemple dans notre projet :

- Un blog peut avoir plusieurs tags
 - Un tag peut être associé à plusieurs blogs
- Cette relation nécessite une table de jonction `blog_tags`.

6.4 Intégrité Référentielle

 Definition

Intégrité Référentielle : Règle qui garantit que les relations entre les tables restent cohérentes. Elle empêche la création d'enregistrements "orphelins" qui référencent des enregistrements inexistants.

 Analogie

Analogie des Références : Si vous avez une lettre qui référence un dossier #123, l'intégrité référentielle garantit que le dossier #123 existe vraiment. Si le dossier est supprimé, la lettre devient une "lettre orpheline".

6.5 Schema Complet de la Base de Donnees

Voici le schema SQL complet de notre base de donnees :

Listing 6.1 – schema.sql - Schema complet de la base de donnees

```

1  -- Activation de l'extension pour les UUID
2  CREATE EXTENSION IF NOT EXISTS pgcrypto;
3
4  --
5  -- TABLE USERS (Utilisateurs)
6  --
7
8  CREATE TABLE users (
9      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
10     email VARCHAR(255) UNIQUE NOT NULL,
11     password_hash VARCHAR(255) NOT NULL,
12     nom VARCHAR(100) NOT NULL,
13     prenom VARCHAR(100) NOT NULL,
14     role VARCHAR(50) DEFAULT 'utilisateur' CHECK (role IN ('utilisateur',
15     'auteur', 'admin')),
16     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
17     date_modification TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
18 );
19 --
20 -- TABLE BLOGS (Articles)
21 --
22
23 CREATE TABLE blogs (
24     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
25     titre VARCHAR(255) NOT NULL,
26     contenu TEXT NOT NULL,
27     auteur_id UUID NOT NULL,
28     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
29     date_modification TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
30 ,
31     CONSTRAINT fk_auteur
32         FOREIGN KEY (auteur_id)
33         REFERENCES users(id)
34         ON DELETE CASCADE
35 );
36 --
37 -- TABLE COMMENTAIRES (Commentaires)
38 --

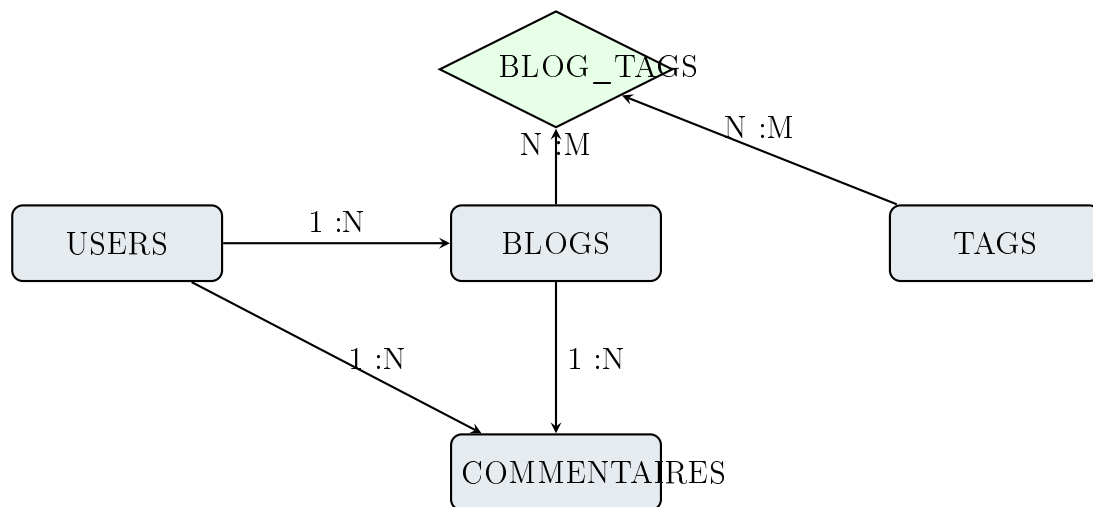
```

```

39
40 CREATE TABLE commentaires (
41     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
42     contenu TEXT NOT NULL,
43     auteur_id UUID NOT NULL,
44     blog_id UUID NOT NULL,
45     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
46     CONSTRAINT fk_auteur_commentaire
47         FOREIGN KEY (auteur_id)
48         REFERENCES users(id)
49         ON DELETE CASCADE,
50     CONSTRAINT fk_blog
51         FOREIGN KEY (blog_id)
52         REFERENCES blogs(id)
53         ON DELETE CASCADE
54 );
55
56 --
57 -- TABLE TAGS (Etiquettes)
58 --
59
60 CREATE TABLE tags (
61     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
62     nom VARCHAR(100) UNIQUE NOT NULL,
63     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
64 );
65
66 --
67 -- TABLE BLOG_TAGS (Relation Many-to-Many)
68 --
69
70 CREATE TABLE blog_tags (
71     blog_id UUID NOT NULL,
72     tag_id UUID NOT NULL,
73     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
74     PRIMARY KEY (blog_id, tag_id),
75     CONSTRAINT fk_blog_tag
76         FOREIGN KEY (blog_id)
77         REFERENCES blogs(id)
78         ON DELETE CASCADE,
79     CONSTRAINT fk_tag_blog
80         FOREIGN KEY (tag_id)
81         REFERENCES tags(id)
82         ON DELETE CASCADE
83 );

```

6.6 Diagramme Entite-Association (ERD)



6.7 Explication des Contraintes

6.7.1 NOT NULL

Definition

NOT NULL : Contrainte qui garantit qu'une colonne ne peut pas contenir de valeur NULL. La colonne doit obligatoirement avoir une valeur.

6.7.2 UNIQUE

Definition

UNIQUE : Contrainte qui garantit que toutes les valeurs dans une colonne sont différentes. Deux lignes ne peuvent pas avoir la même valeur dans cette colonne.

6.7.3 CHECK

Definition

CHECK : Contrainte qui définit une condition booléenne que chaque valeur doit satisfaire. Si la condition est fausse, la valeur est rejetée.

```
1 CHECK (role IN ('utilisateur', 'auteur', 'admin'))
```

Cette contrainte garantit que le rôle ne peut être que l'une de ces trois valeurs.

6.7.4 DEFAULT

Definition

DEFAULT : Valeur par défaut assignée à une colonne si aucune valeur n'est spécifiée lors de l'insertion.

```
1 date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

Cette colonne reçoit automatiquement la date et heure actuelle si aucune valeur n'est fournie.

6.8 Comportement ON DELETE

6.8.1 CASCADE

Definition

ON DELETE CASCADE : Quand un enregistrement référence est supprimé, tous les enregistrements qui le référencent sont également supprimés automatiquement.

Analogie

Analogie : Si vous supprimez un dossier principal, tous les sous-dossiers et fichiers à l'intérieur sont également supprimés automatiquement.

6.8.2 RESTRICT

Definition

ON DELETE RESTRICT : Empêche la suppression d'un enregistrement s'il est référencé par d'autres enregistrements. La suppression est rejetée.

6.8.3 SET NULL

Definition

ON DELETE SET NULL : Quand un enregistrement référence est supprimé, les colonnes de clé étrangère des enregistrements qui le référencent sont mises à NULL.

 Bonne Pratique**Choix du Comportement :**

- **CASCADE** : Quand les enregistrements dependants n'ont pas de sens sans l'enregistrement parent (ex : commentaires sans article)
- **RESTRICT** : Quand la suppression pourrait causer des incoherences (ex : commandes sans client)
- **SET NULL** : Quand la relation est optionnelle (rarement utilise)

Chapitre 7

Connexion Node.js ↔ PostgreSQL

7.1 Introduction au Driver pg

Le driver `pg` est le driver PostgreSQL officiel pour Node.js. Il permet d'exécuter des requêtes SQL depuis Node.js et de gérer les connexions à la base de données.

Definition

Driver de Base de Données : Bibliothèque qui permet à une application de communiquer avec un système de base de données spécifique. Le driver traduit les appels de l'application en protocole de communication de la base de données.

7.2 Fichier de Configuration `db.js`

Créez le fichier `db.js` à la racine du projet :

```
1 const { Pool } = require('pg');
2 require('dotenv').config();
3
4 //
5 // =====
6 // CONFIGURATION DU POOL DE CONNEXIONS
7 // =====
8
9 const pool = new Pool({
10   user: process.env.DB_USER,
11   host: process.env.DB_HOST,
12   database: process.env.DB_NAME,
13   password: process.env.DB_PASSWORD,
14   port: process.env.DB_PORT || 5432,
15   // Configuration du pool
16   max: 20, // Nombre maximum de clients dans le pool
17   min: 2, // Nombre minimum de clients dans le pool
18 })
```

```

17     idleTimeoutMillis: 30000,    // Temps d'inactivite avant fermeture d'
    un client
18     connectionTimeoutMillis: 2000, // Timeout de connexion
19 });
20
21 //
    =====
22 // TEST DE CONNEXION
23 //
    =====
24
25 pool.on('connect', () => {
26     console.log('Nouvelle connexion etablie avec PostgreSQL');
27 });
28
29 pool.on('error', (err) => {
30     console.error('Erreur inattendue sur le client idle', err);
31     process.exit(-1);
32 });
33
34 //
    =====
35 // FONCTION DE TEST DE CONNEXION
36 //
    =====
37
38 const testConnection = async () => {
39     try {
40         const client = await pool.connect();
41         const result = await client.query('SELECT NOW()');
42         console.log('Connexion reussie a PostgreSQL :', result.rows[0]);
43         client.release();
44     } catch (err) {
45         console.error('Erreur de connexion a PostgreSQL :', err);
46         throw err;
47     }
48 };
49
50 // Test de connexion au demarrage
51 testConnection();
52
53 //
    =====
54 // EXPORT DU POOL
55 //
    =====
56
57 module.exports = pool;

```

Listing 7.1 – db.js - Configuration de la connexion PostgreSQL

7.3 Explication du Code

7.3.1 Pool de Connexions

```
1 const { Pool } = require('pg');
```

Definition

Pool de Connexions : Ensemble de connexions a la base de donnees reutilisables. Au lieu de creer une nouvelle connexion pour chaque requete, le pool reutilise les connexions existantes, ce qui ameliore les performances.

Analogie

Analogie de la Piscine : Un pool de connexions est comme une piscine de taxis. Au lieu d'appeler un nouveau taxi a chaque fois, vous reutilisez les taxis disponibles dans la piscine. C'est plus efficace que d'appeler un nouveau taxi a chaque trajet.

Avantages du Pool :

- **Performance** : Reutilise les connexions existantes
- **Scalabilite** : Gere automatiquement le nombre de connexions
- **Fiabilite** : Gere les connexions defaillantes
- **Efficacite** : Evite le surcout de creation de connexions

7.3.2 Configuration du Pool

```
1 max: 20, // Nombre maximum de connexions simultanees
2 min: 2, // Nombre minimum de connexions maintenues
3 idleTimeoutMillis: 30000, // Temps avant fermeture d'une connexion
  inactive
4 connectionTimeoutMillis: 2000, // Temps maximum pour etablir une
  connexion
```

Definition

Configuration du Pool :

- **max** : Nombre maximum de connexions simultanees
- **min** : Nombre minimum de connexions maintenues
- **idleTimeoutMillis** : Temps avant fermeture d'une connexion inactive
- **connectionTimeoutMillis** : Temps maximum pour etablir une connexion

✓ Bonne Pratique

Bonnes Pratiques de Pool :

- max ne doit pas dépasser la limite de connexions de PostgreSQL (default : 100)
- min doit être au moins 1 pour éviter la latence de première connexion
- idleTimeoutMillis doit être adapté à votre trafic
- Toujours utiliser un pool en production

7.3.3 Variables d'Environnement

Créez le fichier `.env` à la racine du projet :

Listing 7.2 – `.env` - Variables d'environnement

```
1 DB_USER=postgres
2 DB_HOST=localhost
3 DB_NAME=blog_db
4 DB_PASSWORD=votre_mot_de_passe
5 DB_PORT=5432
6 PORT=3000
7 ACCESS_TOKEN_SECRET=votre_cle_secrete_access
8 REFRESH_TOKEN_SECRET=votre_cle_secrete_refresh
```

! Attention

Sécurité Critique :

- Le fichier `.env` ne doit **jamais** être commité dans Git
- Ajoutez `.env` dans votre `.gitignore`
- Utilisez des mots de passe forts et des clés secrètes longues
- Changez les valeurs par défaut en production

7.4 Execution de Requetes SQL

7.4.1 Requete Simple

```
1 const db = require('./db');
2
3 async function getAllUsers() {
4   try {
5     const result = await db.query('SELECT * FROM users');
6     return result.rows;
7   } catch (err) {
8     console.error('Erreur:', err);
9     throw err;
10  }
11 }
```

Listing 7.3 – Execution d’une requete simple

7.4.2 Requete avec Parametres

```
1 async function getUserByEmail(email) {  
2   try {  
3     const result = await db.query(  
4       'SELECT * FROM users WHERE email = $1',  
5       [email]  
6     );  
7     return result.rows[0];  
8   } catch (err) {  
9     console.error('Erreur:', err);  
10    throw err;  
11  }  
12 }
```

Listing 7.4 – Requete avec parametres (recommande)

Definition

Parametres Positionnels (\$1, \$2, ...) : Placeholders dans les requetes SQL qui sont remplaces par les valeurs fournies dans un tableau. Cela previent les injections SQL.

7.5 Erreurs Courantes

7.5.1 ECONNREFUSED

! Erreur

Erreur : connection refused at TCPConnectWrap.afterConnect
Cause : PostgreSQL n’est pas demarre ou n’ecoute pas sur le port specifie.
Diagnostic :

- Verifiez que PostgreSQL est installe
- Verifiez que le service PostgreSQL est demarre
- Verifiez le port dans `.env`

Resolution :

```
1 # Sous Linux/Mac  
2 sudo service postgresql start  
3  
4 # Sous Windows  
5 # Demarrer le service PostgreSQL via le gestionnaire de services
```

7.5.2 Password Authentication Failed

! Erreur

Erreur : password authentication failed for user "postgres"

Cause : Le mot de passe dans `.env` est incorrect.

Resolution : Sur Windows, il est recommande d'utiliser l'interface graphique `pgAdmin` (fournie avec l'installateur PostgreSQL) ou l'application "SQL Shell (`psql`)" plutôt que d'exécuter directement des commandes dans l'invite Windows.

Procédure (`pgAdmin`) :

- Ouvrir `pgAdmin` et se connecter au serveur en tant que `postgres`.
- Ouvrir le *Query Tool* pour le serveur ou la base ciblée.
- Exécuter la commande SQL suivante dans le *Query Tool* :

```
1 ALTER USER postgres WITH PASSWORD 'nouveau_mot_de_passe';
```

Alternatives en ligne de commande (si souhaité) :

```
1 # Linux / macOS :
2 sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD '
   nouveau_mot_de_passe';"
3
4 # Windows (appel direct du binaire psql fourni par PostgreSQL) :
5 "C:\\Program Files\\PostgreSQL\\15\\bin\\psql.exe" -U postgres -c "
   ALTER USER postgres WITH PASSWORD 'nouveau_mot_de_passe';"
```

7.6 Creation de la Base de Donnees

Sur Windows, il est plus simple d'utiliser `pgAdmin` pour créer une base : ouvrir `pgAdmin` → se connecter au serveur `postgres` → cliquer sur *Databases* → *Create* → *Database*. Vous pouvez aussi

```
1 CREATE DATABASE blog_db;
```

Alternatives en ligne de commande :

Listing 7.5 – Creation de la base de donnees PostgreSQL

```
1 # Linux / macOS :
2 sudo -u postgres psql -c "CREATE DATABASE blog_db;"
3
4 # Windows (appel direct du binaire psql) :
5 "C:\\Program Files\\PostgreSQL\\15\\bin\\psql.exe" -U postgres -c "
   CREATE DATABASE blog_db;"
```

7.7 Execution du Script SQL

Pour exécuter un fichier SQL depuis `pgAdmin` : ouvrir la base `blog_db`, ouvrir le *Query Tool*, puis charger le fichier SQL. Alternatives en ligne de commande :

Listing 7.6 – Execution du script schema.sql

```
1 # Linux / macOS
2 psql -U postgres -d blog_db -f sql/schema.sql
3
4 # Windows (chemin Windows)
5 "C:\\Program Files\\PostgreSQL\\15\\bin\\psql.exe" -U postgres -d
   blog_db -f sql\\schema.sql
```

✓ Bonne Pratique

Bonnes Pratiques de Base de Données :

- Toujours utiliser des variables d'environnement pour les credentials
- Utiliser des parametres dans les requetes (jamais de concatenation)
- Gerer les erreurs de connexion gracieusement
- Utiliser un pool de connexions en production
- Ne jamais exposer les erreurs SQL detaillees au client

7.8 Connexion à PostgreSQL depuis pgAdmin et Node

7.8.1 Comment savoir / réinitialiser le mot de passe utilisé par pgAdmin

Sur Windows, pgAdmin se connecte au serveur PostgreSQL en utilisant les identifiants du compte PostgreSQL (souvent l'utilisateur 'postgres') et le mot de passe choisi lors de l'installation. Si vous ne vous rappelez plus du mot de passe, procédez comme suit :

- Méthode recommandée (ligne de commande) :
 - Ouvrir une Invite de commandes / PowerShell élevée (exécuter en tant qu'administrateur).
 - Appeler le binaire 'psql' fourni par PostgreSQL et exécuter une commande SQL pour réinitialiser le mot de passe :

```
1 "C:\\Program Files\\PostgreSQL\\15\\bin\\psql.exe" -U postgres -c "
   ALTER USER postgres WITH PASSWORD 'NouveauMotDePasse';"
```

- Si vous avez accès à une machine Linux/macOS où PostgreSQL est installé :

```
1 sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD '
   NouveauMotDePasse';"
```

- Une fois le mot de passe réinitialisé, ouvrez pgAdmin et entrez le nouveau mot de passe dans la configuration de la connexion (clic droit sur le serveur → *Properties* → onglet *Connection*).

- Si vous ne pouvez pas exécuter 'psql' localement, vous pouvez démarrer PostgreSQL en mode single-user ou utiliser l'installateur pour réparer/réinitialiser mais ces méthodes sont avancées : préférez la commande ci-dessus.

7.8.2 Quelles informations entrer dans pgAdmin et dans '.env' pour Node

Pour connecter votre application Node.js à PostgreSQL (et pour configurer pgAdmin), vous aurez besoin des informations suivantes :

- Host : adresse du serveur (ex. 'localhost' ou une IP)
- Port : port PostgreSQL (par défaut '5432')
- Database : nom de la base (ex. 'blog_db')
- User : nom d'utilisateur PostgreSQL (ex. 'postgres')
- Password : mot de passe PostgreSQL (celui réinitialisé ci-dessus)
- SSL : si votre serveur nécessite TLS/SSL (souvent 'true' en prod)

Exemple de fichier '.env' pour Node :

```
1 DB_HOST=localhost
2 DB_PORT=5432
3 DB_NAME=blog_db
4 DB_USER=postgres
5 DB_PASSWORD=VotreMotDePasse
6 # Optionnel : connection string
7 # DATABASE_URL=postgres://postgres:VotreMotDePasse@localhost:5432/
  blog_db
8 DB_SSL=false
```

Exemple minimal d'utilisation dans Node (package 'pg') :

```
1 // npm install pg
2 const { Pool } = require('pg');
3
4 const pool = new Pool({
5   host: process.env.DB_HOST,
6   port: parseInt(process.env.DB_PORT || '5432', 10),
7   user: process.env.DB_USER,
8   password: process.env.DB_PASSWORD,
9   database: process.env.DB_NAME,
10  ssl: process.env.DB_SSL === 'true'
11 });
12
13 async function test() {
14   const res = await pool.query('SELECT NOW()');
15   console.log(res.rows[0]);
16   await pool.end();
17 }
18
19 test().catch(err => console.error(err));
```

Listing 7.7 – Connexion Node.js avec pg

Conseils pratiques :

- Ne stockez jamais de mots de passe en clair dans le code source : utilisez des variables d'environnement ou un gestionnaire de secrets.
- Vérifiez que PostgreSQL accepte les connexions depuis l'hôte indiqué en contrôlant 'listen_addresses'dans'postgresql.conf'etlesrglesde'pg_hba.conf'.

7.9 Authentification : Access token et Refresh token

Pour des sessions sécurisées, on utilise typiquement des tokens JWT d'accès (short-lived "access tokens") et des tokens de rafraîchissement (long-lived "refresh tokens"). Le schéma typique :

- L'API émet un 'accessToken' (p. ex. 15 minutes) et un 'refreshToken' (p. ex. 7 jours) à la connexion ('/login').
- Le client utilise 'accessToken' dans l'en-tête 'Authorization: Bearer ...' pour accéder aux ressources protégées.
- Quand l'accessToken expire, le client appelle '/token' avec le refreshToken pour obtenir un nouveau accessToken.
- Le refreshToken peut être révoqué côté serveur lors de la déconnexion.

Ci-dessous un exemple minimal en Node.js (Express) montrant l'implémentation des endpoints 'POST /login', 'POST /token' et 'POST /logout', ainsi qu'un middleware 'authenticateToken' pour protéger les routes. Le code complet se trouve dans les fichiers d'exemple 'server.js' et 'auth.js' fournis avec ce projet.

```
1 // Extrait de server.js (version compl te dans workspace)
2 const express = require('express');
3 const jwt = require('jsonwebtoken');
4 const bcrypt = require('bcrypt');
5 const { generateAccessToken, generateRefreshToken } = require('./auth');
6
7 // stockage m moire des refresh tokens (pour d mo). En prod : stocker
  en DB.
8 let refreshTokens = [];
9
10 app.post('/login', async (req, res) => {
11   // v rifier utilisateur / mot de passe (hash)
12   const payload = { id: user.id, username: user.username };
13   const accessToken = generateAccessToken(payload);
14   const refreshToken = generateRefreshToken(payload);
15   refreshTokens.push(refreshToken);
16   res.json({ accessToken, refreshToken });
17 });
18
19 app.post('/token', (req, res) => {
20   const { refreshToken } = req.body;
21   if (!refreshToken) return res.sendStatus(401);
22   if (!refreshTokens.includes(refreshToken)) return res.sendStatus
    (403);
23   jwt.verify(refreshToken, process.env.REFRESH_TOKEN_SECRET, (err,
    user) => {
24     if (err) return res.sendStatus(403);
```

```
25     const accessToken = generateAccessToken({ id: user.id, username:
26       user.username });
27     res.json({ accessToken });
28   });
29
30 app.post('/logout', (req, res) => {
31   const { refreshToken } = req.body;
32   refreshTokens = refreshTokens.filter(t => t !== refreshToken);
33   res.sendStatus(204);
34 });
```

Listing 7.8 – Exemple minimal d'authentification avec refresh tokens

Le code d'exemple complet est fourni dans les fichiers créés : 'server.js', 'auth.js', un 'package.json' minimal et '.env.example'.

Chapitre 8

Securite des Mots de Passe — Bcrypt

8.1 Introduction a la Securite des Mots de Passe

La securite des mots de passe est l'un des aspects les plus critiques de la securite des applications. Une faille dans ce domaine peut avoir des consequences catastrophiques.

Securite

Statistiques Effrayantes :

- 81% des violations de donnees sont dues a des mots de passe faibles ou reutilises
- 59% des personnes utilisent le meme mot de passe partout
- Une attaque par force brute peut tester des milliards de mots de passe par seconde

8.2 Pourquoi Ne Pas Stocker les Mots de Passe en Clair ?

Erreur

JAMAIS stocker les mots de passe en clair !

Si vous stockez les mots de passe en clair dans votre base de données :

- Toute personne ayant accès à la base de données peut voir tous les mots de passe
- En cas de fuite de données, tous les mots de passe sont compromis
- Les attaquants peuvent utiliser ces mots de passe sur d'autres sites (réutilisation)
- C'est une violation grave de la confidentialité et de la confiance

Analogie

Analogie du Coffre-fort : Stocker un mot de passe en clair est comme écrire votre code PIN sur votre carte bancaire. Si quelqu'un vole votre carte, il a immédiatement accès à votre compte.

8.3 Hachage vs Chiffrement

8.3.1 Chiffrement (Encryption)

Definition

Chiffrement : Processus de transformation de données lisibles en données illisibles à l'aide d'une clé. Le chiffrement est réversible : avec la bonne clé, on peut retrouver les données originales.

Analogie

Analogie du Coffre : Le chiffrement est comme mettre un document dans un coffre avec une clé. Avec la clé, vous pouvez récupérer le document original.

8.3.2 Hachage (Hashing)

Definition

Hachage : Processus de transformation de données en une empreinte de taille fixe. Le hachage est irréversible : il est mathématiquement impossible de retrouver les données originales à partir du hash.

Analogie

Analogie de l'Empreinte Digitale : Le hachage est comme prendre une empreinte digitale. Vous pouvez créer une empreinte à partir d'un doigt, mais vous ne pouvez pas recréer le doigt à partir de l'empreinte.

Bonne Pratique

Pour les Mots de Passe : TOUJOURS utiliser le hachage, jamais le chiffrement
Les mots de passe ne doivent jamais être récupérables, même par l'administrateur. Le hachage garantit cette irréversibilité.

8.4 Le Probleme du Hachage Simple

```
1 const crypto = require('crypto');
2
3 function hashSimple(password) {
4   return crypto.createHash('sha256').update(password).digest('hex');
5 }
6
7 // Exemple
8 console.log(hashSimple('password123'));
9 // Resultat :
   ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Listing 8.1 – Hachage simple (DECONSEILLE)

8.4.1 Probleme : Rainbow Tables

Definition

Rainbow Table : Table précalculée de hachages pour des mots de passe courants. Les attaquants utilisent ces tables pour inverser les hachages instantanément.

 Analogie

Analogie du Dictionnaire Inverse : Une rainbow table est comme un dictionnaire qui liste tous les mots avec leurs traductions. Si vous avez le mot traduit, vous pouvez instantanément trouver le mot original.

Exemple : Le hash de "password123" est toujours le meme. Un attaquant avec une rainbow table peut instantanément savoir que le mot de passe est "password123".

8.5 La Solution : Le Salting

 Definition

Sel (Salt) : Chaîne aleatoire unique ajoutée au mot de passe avant le hachage. Le sel est stocké avec le hash et rend les rainbow tables inutiles.

 Analogie

Analogie du Poivre : Le sel est comme ajouter du poivre unique à chaque plat. Même si deux plats utilisent les mêmes ingrédients de base, le poivre unique rend chaque plat différent.

```
1 const crypto = require('crypto');
2
3 function hashWithSalt(password, salt) {
4   return crypto.createHash('sha256')
5     .update(password + salt)
6     .digest('hex');
7 }
8
9 // Generation d'un sel aleatoire
10 const salt = crypto.randomBytes(16).toString('hex');
11 const hash = hashWithSalt('password123', salt);
```

Listing 8.2 – Hachage avec sel (manuel - DECONSEILLE)

8.6 Bcrypt : La Solution Professionnelle

Bcrypt est une bibliothèque conçue spécifiquement pour le hachage de mots de passe. Elle combine :

- Hachage irréversible
- Salting automatique
- Lenteur configurable (pour ralentir les attaques)

8.6.1 Fonctionnement de Bcrypt

```
1 const bcrypt = require('bcrypt');
2
3 async function hashPassword(password) {
4   const saltRounds = 10;
5   const hash = await bcrypt.hash(password, saltRounds);
6   return hash;
7 }
8
9 // Exemple
10 hashPassword('password123').then(hash => {
11   console.log(hash);
12   // $2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
13 });
```

Listing 8.3 – Hachage avec bcrypt

8.6.2 Structure du Hash Bcrypt

Definition

Structure du Hash Bcrypt : \$2b\$10\$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17

- \$2b\$: Identifiant de l'algorithme bcrypt
- 10 : Cout de calcul (nombre de rounds)
- N9qo8uL0ickgx2ZMRZoMye : Sel (22 caracteres)
- IjZAgcf17p92ldGxad68LJZdL17lhWy : Hash (31 caracteres)

8.6.3 Salt Rounds (Cout de Calcul)

Definition

Salt Rounds : Parametre qui controle le nombre d'iterations de hachage. Plus le nombre est eleve, plus le hachage est lent (et securise), mais plus il consomme de ressources.

- 10 : Valeur recommandee actuelle (100ms)
- 12 : Plus securise (400ms)
- 8 : Plus rapide (50ms)

 Bonne Pratique

Choix des Salt Rounds :

- 10 est un bon compromis securite/performance
- Ajustez selon vos besoins et votre infrastructure
- Testez le temps de hachage sur votre serveur
- Ne descendez jamais en dessous de 8

8.7 Verification d'un Mot de Passe

```
1 const bcrypt = require('bcrypt');
2
3 async function verifyPassword(password, hash) {
4   const isValid = await bcrypt.compare(password, hash);
5   return isValid;
6 }
7
8 // Exemple
9 const hash = '
10   $2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy';
11
12 verifyPassword('password123', hash).then(isValid => {
13   console.log(isValid); // true
14 });
15
16 verifyPassword('wrongpassword', hash).then(isValid => {
17   console.log(isValid); // false
18 });
```

Listing 8.4 – Verification d'un mot de passe avec bcrypt

 Definition

`bcrypt.compare()` : Fonction qui compare un mot de passe en clair avec un hash bcrypt. Elle extrait automatiquement le sel du hash et effectue la comparaison securisee.

8.8 Implementation dans l'Application

Creez le fichier `routes/auth.js` :

```
1 const express = require('express');
2 const bcrypt = require('bcrypt');
3 const db = require('../db');
4
5 const router = express.Router();
6
```

```
7 //
8 // =====
9 // ROUTE D'INSCRIPTION
10 // =====
11 router.post('/register', async (req, res) => {
12   try {
13     const { email, password, nom, prenom } = req.body;
14
15     // Validation des donnees
16     if (!email || !password || !nom || !prenom) {
17       return res.status(400).json({
18         error: 'Tous les champs sont obligatoires'
19       });
20     }
21
22     // Validation de l'email
23     const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
24     if (!emailRegex.test(email)) {
25       return res.status(400).json({
26         error: 'Email invalide'
27       });
28     }
29
30     // Validation du mot de passe
31     if (password.length < 8) {
32       return res.status(400).json({
33         error: 'Le mot de passe doit contenir au moins 8
34 caracteres'
35       });
36     }
37
38     // Verification si l'email existe deja
39     const existingUser = await db.query(
40       'SELECT id FROM users WHERE email = $1',
41       [email]
42     );
43
44     if (existingUser.rows.length > 0) {
45       return res.status(409).json({
46         error: 'Cet email est deja utilise'
47       });
48     }
49
50     // Hachage du mot de passe
51     const saltRounds = 10;
52     const password_hash = await bcrypt.hash(password, saltRounds);
53
54     // Insertion de l'utilisateur
55     const result = await db.query(
56       'INSERT INTO users (email, password_hash, nom, prenom)
57       VALUES ($1, $2, $3, $4)
58       RETURNING id, email, nom, prenom, role, date_creation',
```

```
58     [email, password_hash, nom, prenom]
59   );
60
61   const user = result.rows[0];
62
63   // Reponse
64   res.status(201).json({
65     message: 'Inscription reussie',
66     user: {
67       id: user.id,
68       email: user.email,
69       nom: user.nom,
70       prenom: user.prenom,
71       role: user.role
72     }
73   });
74
75   } catch (err) {
76     console.error('Erreur lors de l\'inscription:', err);
77     res.status(500).json({
78       error: 'Erreur lors de l\'inscription'
79     });
80   }
81 });
82
83 module.exports = router;
```

Listing 8.5 – routes/auth.js - Route d'inscription avec bcrypt

8.9 Bonnes Pratiques de Securite des Mots de Passe

✓ Bonne Pratique

Regles d'Or :

- JAMAIS stocker les mots de passe en clair
- TOUJOURS utiliser bcrypt ou Argon2
- TOUJOURS utiliser un sel unique par mot de passe
- TOUJOURS valider la force des mots de passe
- JAMAIS limiter arbitrairement la longueur des mots de passe
- JAMAIS renvoyer le mot de passe (meme hache) dans les reponses API

Securite

Validation des Mots de Passe :

- Minimum 8 caracteres (recommande : 12+)
- Melange de lettres, chiffres et caracteres speciaux
- Interdiction des mots de passe courants ("password", "123456")
- Verification contre les bases de donnees de mots de passe compromis

Chapitre 9

Authentication JWT

9.1 Introduction a l'Authentication

Definition

Authentication : Processus de verification de l'identite d'un utilisateur. L'authentification repond a la question "Qui etes-vous ?".

Analogie

Analogie du Badge : L'authentification est comme presenter votre badge d'employe a l'entree du bureau. Le gardien verifie que le badge est valide et vous laisse entrer.

9.2 Authentication vs Autorisation

Definition

Autorisation : Processus de determination des droits d'accès d'un utilisateur authentifie. L'autorisation repond a la question "Qu'avez-vous le droit de faire ?".

- Authentication : "Qui etes-vous ?" (ex: login/mot de passe)
- Autorisation : "Qu'avez-vous le droit de faire ?" (ex: role admin vs utilisateur)

9.3 Pourquoi JWT ?

9.3.1 Authentification Traditionnelle avec Sessions

Definition

Session : Stockage cote serveur des informations d'authentification. Le serveur cree une session et envoie un cookie de session au client.

Problemes des Sessions :

- Stateful : Le serveur doit stocker les sessions (memoire ou base de donnees)
- Scalabilite : Difficile de scaler horizontalement (plusieurs serveurs)
- Cookies : Necessite des cookies (vulnerabilites CSRF)
- Nettoyage : Les sessions expirees doivent etre nettoyees

9.3.2 Authentification Stateless avec JWT

Definition

Stateless : Le serveur ne stocke aucune information d'authentification. Toutes les informations necessaires sont contenues dans le token JWT lui-meme.

Avantages de JWT :

- Stateless : Pas de stockage cote serveur
- Scalabilite : Facile de scaler horizontalement
- Cross-domain : Fonctionne sur plusieurs domaines
- Mobile-friendly : Facile a utiliser sur mobile
- Standard : Standard industriel (RFC 7519)

9.4 Structure d'un JWT

Un JWT est compose de trois parties separees par des points :

```
1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
  eyJ1c2VySWQiOiIxMjMONTY3ODkwIiwiaWF0IjoxNTE2MjM5MDIyfQ.  
  SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

9.4.1 Header (En-tete)

```
1 {  
2   "alg": "HS256",  
3   "typ": "JWT"  
4 }
```

Definition

Header : Contient les metadonnees du token (algorithme de signature, type de token).

9.4.2 Payload (Charge Utile)

```
1 {  
2   "userId": "1234567890",  
3   "email": "user@example.com",  
4   "role": "auteur",  
5   "iat": 1516239022  
6 }
```

Definition

Payload : Contient les claims (informations) sur l'utilisateur. Les claims standards incluent :

- iss (issuer) : Emetteur du token
- sub (subject) : Sujet du token
- aud (audience) : Audience du token
- exp (expiration) : Date d'expiration
- iat (issued at) : Date d'emission

Attention

Attention : Le payload est encode en Base64, pas chiffre. N'importe qui peut decoder et lire le contenu. Ne stockez JAMAIS d'informations sensibles (mots de passe, numeros de carte) dans le payload.

9.4.3 Signature

Definition

Signature : Signature cryptographique qui garantit l'integrite et l'authenticite du token. Elle est generee en signant le header et le payload avec une cle secrete.

```
1 HMACSHA256(  
2   base64UrlEncode(header) + "." + base64UrlEncode(payload),  
3   secret  
4 )
```

9.5 Implementation de la Connexion

Ajoutez la route de connexion dans routes/auth.js :

```
1 const jwt = require('jsonwebtoken');
2
3 //
4 // ROUTE DE CONNEXION
5 //
6
7 router.post('/login', async (req, res) => {
8   try {
9     const { email, password } = req.body;
10
11     // Validation des donnees
12     if (!email || !password) {
13       return res.status(400).json({
14         error: 'Email et mot de passe obligatoires'
15       });
16     }
17
18     // Recherche de l'utilisateur
19     const result = await db.query(
20       'SELECT * FROM users WHERE email = $1',
21       [email]
22     );
23
24     if (result.rows.length === 0) {
25       return res.status(401).json({
26         error: 'Email ou mot de passe incorrect'
27       });
28     }
29
30     const user = result.rows[0];
31
32     // Verification du mot de passe
33     const isValidPassword = await bcrypt.compare(password, user.
password_hash);
34
35     if (!isValidPassword) {
36       return res.status(401).json({
37         error: 'Email ou mot de passe incorrect'
38       });
39     }
40
41     // Generation des tokens JWT
42     const accessToken = jwt.sign(
43       {
44         userId: user.id,
45         email: user.email,
46         role: user.role
47       },
```

```
48     process.env.ACCESS_TOKEN_SECRET,
49     { expiresIn: '15m' }
50   );
51
52   const refreshToken = jwt.sign(
53     {
54       userId: user.id,
55       email: user.email,
56       role: user.role
57     },
58     process.env.REFRESH_TOKEN_SECRET,
59     { expiresIn: '7d' }
60   );
61
62   // En production, stockez le refresh token dans la base de
63   // données ou un cache
64   // pour pouvoir le récupérer en cas de déconnexion ou de
65   // compromission.
66   res.json({
67     message: 'Connexion réussie',
68     accessToken,
69     refreshToken,
70     user: {
71       id: user.id,
72       email: user.email,
73       nom: user.nom,
74       prenom: user.prenom,
75       role: user.role
76     }
77   });
78
79   } catch (err) {
80     console.error('Erreur lors de la connexion:', err);
81     res.status(500).json({
82       error: 'Erreur lors de la connexion'
83     });
84   }
85 });
86
87 // Route de rafraîchissement du token
88 router.post('/token', async (req, res) => {
89   const { refreshToken } = req.body;
90
91   if (!refreshToken) {
92     return res.status(400).json({ error: 'Refresh token manquant' });
93   }
94
95   try {
96     const decoded = jwt.verify(refreshToken, process.env.
97     REFRESH_TOKEN_SECRET);
98
99     const accessToken = jwt.sign(
100       { userId: decoded.userId, email: decoded.email, role:
101       decoded.role },
102       process.env.ACCESS_TOKEN_SECRET,
```

```
99         { expiresIn: '15m' }
100     );
101
102     res.json({ accessToken });
103 } catch (err) {
104     return res.status(401).json({ error: 'Refresh token invalide ou
105 expire' });
106 }
107 });
108 // Route de deconnexion
109 router.post('/logout', (req, res) => {
110     // En production, invalidez le refresh token dans la base ou
111     supprimez-le du store.
112     res.json({ message: 'Deconnexion effectuee' });
113 });
```

Listing 9.1 – routes/auth.js - Route de connexion avec JWT

9.6 Generation du Token

```
1 const accessToken = jwt.sign(
2   {
3     userId: user.id,
4     email: user.email,
5     role: user.role
6   },
7   process.env.ACCESS_TOKEN_SECRET,
8   { expiresIn: '15m' }
9 );
10
11 const refreshToken = jwt.sign(
12   {
13     userId: user.id,
14     email: user.email,
15     role: user.role
16   },
17   process.env.REFRESH_TOKEN_SECRET,
18   { expiresIn: '7d' }
19 );
```

Definition

`jwt.sign()` : Fonction qui genere un token JWT. Elle prend trois arguments :

1. Le payload (donnees a inclure dans le token)
2. La cle secrete (pour signer le token)
3. Les options (ex: expiration)

9.7 Securite de la Cle Secrete

Securite

Regles d'Or pour JWT_SECRET :

- Utilisez une cle secrete longue et aleatoire (minimum 32 caracteres)
- Ne JAMAIS hardcode la cle secrete dans le code
- Utilisez des cles differentes pour developpement et production
- Changez la cle regulierement (rotation des cles)
- Stockez la cle dans un gestionnaire de secrets en production

```
1 # Generation d'une cle secrete securisee
2 node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

9.8 Expiration du Token

Definition

Expiration : Date a laquelle le token n'est plus valide. Apres cette date, le token est rejete.

Valeurs courantes d'expiration :

- 15 minutes : Tres securise (necessite refresh token)
- 1 heure : Securise (standard)
- 24 heures : Moderement securise
- 7 jours : Moins securise

Bonne Pratique

Recommandation : Utilisez une expiration courte (15-60 minutes) avec un refresh token pour les applications sensibles. Pour ce projet pedagogique, nous utilisons 24 heures.

Chapitre 10

Les Middlewares Express

10.1 Introduction aux Middlewares

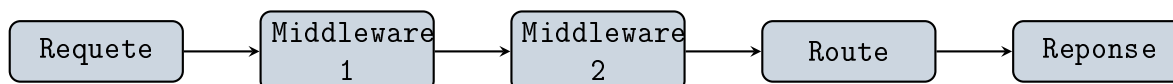
Definition

Middleware : Fonction qui a acces a l'objet requete (req), l'objet reponse (res), et la fonction middleware suivante (next) dans le cycle requete-reponse de l'application.

```
1 function middleware(req, res, next) {  
2   // Traitement de la requete  
3   console.log('Middleware execute');  
4  
5   // Passage au middleware suivant  
6   next();  
7 }
```

10.2 Cycle de Vie d'une Requete

1. La requete arrive au serveur
2. Les middlewares globaux s'executent dans l'ordre
3. Les middlewares specifiques a la route s'executent
4. Le gestionnaire de route s'execute
5. La reponse est envoyee



10.3 Types de Middlewares

10.3.1 Middlewares au Niveau Application

```
1 // S'applique a toutes les routes
2 app.use(express.json());
3 app.use(cors());
```

10.3.2 Middlewares au Niveau Routeur

```
1 // S'applique a toutes les routes du routeur
2 router.use(authMiddleware);
```

10.3.3 Middlewares au Niveau Route

```
1 // S'applique uniquement a cette route
2 router.get('/protected', authMiddleware, (req, res) => {
3     res.json({ message: 'Route protegee' });
4 });
```

10.3.4 Middlewares de Gestion d'Erreurs

```
1 // 4 arguments = middleware d'erreur
2 app.use((err, req, res, next) => {
3     console.error(err);
4     res.status(500).json({ error: 'Erreur interne' });
5 });
```

10.4 Middleware d'Authentification JWT

Creez le fichier middleware/auth.js :

```
1 const jwt = require('jsonwebtoken');
2
3 const authMiddleware = (req, res, next) => {
4     try {
5         // Recuperation du token depuis l'en-tete Authorization
6         const token = req.header('Authorization')?.replace('Bearer ', '');
7
8         // Verification de la presence du token
9         if (!token) {
10             return res.status(401).json({
11                 error: 'Token d\'authentification manquant'
12             });
13         }
14
15         // Verification et decodage du token d'accès
16         const decoded = jwt.verify(token, process.env.ACCESS_TOKEN_SECRET);
17
18         // Ajout des informations utilisateur a la requete
```

```
19     req.user = decoded;
20
21     // Passage au middleware suivant
22     next();
23
24   } catch (err) {
25     if (err.name === 'TokenExpiredError') {
26       return res.status(401).json({
27         error: 'Token expire'
28       });
29     }
30
31     if (err.name === 'JsonWebTokenError') {
32       return res.status(401).json({
33         error: 'Token invalide'
34       });
35     }
36
37     console.error('Erreur d\'authentification:', err);
38     res.status(500).json({
39       error: 'Erreur lors de l\'authentification'
40     });
41   }
42 };
43
44 module.exports = authMiddleware;
```

Listing 10.1 – middleware/auth.js - Middleware d'authentification JWT

10.5 Middleware de Verification des Roles

Creez le fichier middleware/roles.js :

```
1 const checkRole = (...allowedRoles) => {
2   return (req, res, next) => {
3     // Verification que l'utilisateur est authentifie
4     if (!req.user) {
5       return res.status(401).json({
6         error: 'Utilisateur non authentifie'
7       });
8     }
9
10    // Verification que le role est autorise
11    if (!allowedRoles.includes(req.user.role)) {
12      return res.status(403).json({
13        error: 'Acces non autorise'
14      });
15    }
16
17    // Passage au middleware suivant
18    next();
19  };
20 };
21
```

```
22 module.exports = { checkRole };
```

Listing 10.2 – middleware/roles.js - Middleware de verification des roles

Definition

Higher-Order Function : `checkRole` est une fonction qui retourne une fonction middleware. Cela permet de passer des parametres au middleware.

10.6 Utilisation des Middlewares

```
1 const authMiddleware = require('../middleware/auth');
2 const { checkRole } = require('../middleware/roles');
3
4 // Route protegee (authentification requise)
5 router.get('/profile', authMiddleware, async (req, res) => {
6     // req.user contient les informations du token
7     res.json({ user: req.user });
8 });
9
10 // Route admin (role admin requis)
11 router.delete('/users/:id',
12     authMiddleware,
13     checkRole('admin'),
14     async (req, res) => {
15         // Seuls les admins peuvent acceder
16         res.json({ message: 'Utilisateur supprime' });
17     }
18 );
19
20 // Route auteur ou admin
21 router.post('/blogs',
22     authMiddleware,
23     checkRole('auteur', 'admin'),
24     async (req, res) => {
25         // Auteurs et admins peuvent creer des blogs
26         res.json({ message: 'Blog cree' });
27     }
28 );
```

Listing 10.3 – Utilisation des middlewares dans les routes

10.7 Bonnes Pratiques de Middlewares

✓ Bonne Pratique

Regles d'Or :

- Toujours appeler `next()` ou envoyer une reponse
- Ne jamais oublier `next()` dans les middlewares
- Les middlewares doivent etre idempotents (pas d'effets de bord)
- Gerer les erreurs proprement
- Logger les actions importantes

Chapitre 11

Routes Backend Completes

11.1 Introduction aux Routes

Definition

Route : Chemin URL qui correspond a une fonction de gestionnaire specifique. Quand une requete correspond au chemin et a la methode HTTP, le gestionnaire associe est execute.

11.2 Routes d'Authentification

11.2.1 Route d'Inscription

```
1 router.post('/register', async (req, res) => {
2   // Validation des donnees
3   const { email, password, nom, prenom } = req.body;
4
5   // Verification si l'email existe deja
6   const existingUser = await db.query(
7     'SELECT id FROM users WHERE email = $1',
8     [email]
9   );
10
11   if (existingUser.rows.length > 0) {
12     return res.status(409).json({ error: 'Email deja utilise' });
13   }
14
15   // Hachage du mot de passe
16   const password_hash = await bcrypt.hash(password, 10);
17
18   // Insertion de l'utilisateur
19   const result = await db.query(
20     'INSERT INTO users (email, password_hash, nom, prenom)
21     VALUES ($1, $2, $3, $4)
22     RETURNING id, email, nom, prenom, role',
23     [email, password_hash, nom, prenom]
24   );
```

```
25
26     res.status(201).json({
27         message: 'Inscription reussie',
28         user: result.rows[0]
29     });
30 });
```

Listing 11.1 – POST /api/auth/register

11.2.2 Route de Connexion

```
1 router.post('/login', async (req, res) => {
2     const { email, password } = req.body;
3
4     // Recherche de l'utilisateur
5     const result = await db.query(
6         'SELECT * FROM users WHERE email = $1',
7         [email]
8     );
9
10    if (result.rows.length === 0) {
11        return res.status(401).json({ error: 'Identifiants incorrects'
12    });
13    }
14
15    const user = result.rows[0];
16
17    // Verification du mot de passe
18    const isValid = await bcrypt.compare(password, user.password_hash);
19
20    if (!isValid) {
21        return res.status(401).json({ error: 'Identifiants incorrects'
22    });
23    }
24
25    // Generation des tokens d'accès et de rafraichissement
26    const accessToken = jwt.sign(
27        { userId: user.id, email: user.email, role: user.role },
28        process.env.ACCESS_TOKEN_SECRET,
29        { expiresIn: '15m' }
30    );
31
32    const refreshToken = jwt.sign(
33        { userId: user.id, email: user.email, role: user.role },
34        process.env.REFRESH_TOKEN_SECRET,
35        { expiresIn: '7d' }
36    );
37
38    res.json({
39        message: 'Connexion reussie',
40        accessToken,
41        refreshToken,
42        user: { id: user.id, email: user.email, role: user.role }
43    });
44 }
```

```
42 });
```

Listing 11.2 – POST /api/auth/login

11.3 Routes des Blogs

Creez le fichier routes/blogs.js :

```
1 const express = require('express');
2 const db = require('../db');
3 const authMiddleware = require('../middleware/auth');
4 const { checkRole } = require('../middleware/roles');
5
6 const router = express.Router();
7
8 //
9 // =====
10 // RECUPERER TOUS LES BLOGS
11 // =====
12
13 router.get('/', async (req, res) => {
14   try {
15     const result = await db.query(`
16       SELECT b.*, u.nom, u.prenom, u.email as auteur_email,
17       COUNT(c.id) as nombre_commentaires
18       FROM blogs b
19       LEFT JOIN users u ON b.auteur_id = u.id
20       LEFT JOIN commentaires c ON b.id = c.blog_id
21       GROUP BY b.id, u.nom, u.prenom, u.email
22       ORDER BY b.date_creation DESC
23     `);
24     res.json({ blogs: result.rows });
25   } catch (err) {
26     console.error('Erreur:', err);
27     res.status(500).json({ error: 'Erreur lors de la recuperation
28       des blogs' });
29   }
30 });
31 \subsection{Explication pas-a-pas de la requete}
32
33 La requete ci-dessus combine plusieurs concepts SQL courants. Nous
34 expliquons chacun et donnons de petits exemples.
35
36 \begin{itemize}
37   \item \textbf{SELECT b.* , ...} : selectionne toutes les colonnes de
38     la table \texttt{blogs} (alias \texttt{b}) et ajoute des colonnes
39     calculees ou provenant d'autres tables.
40   \item \textbf{LEFT JOIN users u ON b.auteur_id = u.id} : joint la
41     table \texttt{users} a \texttt{blogs}. LEFT JOIN signifie "garder
42     toutes les lignes de la table de gauche (blogs)" meme si aucune
```

```

correspondance n'existe dans \texttt{users} (les colonnes de \texttt{
users} seront NULL si pas de match).
37 \item \textbf{LEFT JOIN commentaires c ON b.id = c.blog\_id} : joint
les commentaires associes a chaque blog. Un blog sans commentaires
reste presente avec NULLs pour \texttt{c}.
38 \item \textbf{COUNT(c.id) as nombre\_commentaires} : compte le
nombre de lignes de la jointure pour chaque blog (nombre de
commentaires). COUNT(ignore les NULLs quand on compte une colonne
specifique).
39 \item \textbf{GROUP BY b.id, u.nom, u.prenom, u.email} : necessaire
parce qu'on utilise une fonction d'aggregation (COUNT). On indique
les colonnes non-aggregees qui doivent apparaître dans le SELECT.
40 \item \textbf{ORDER BY b.date\_creation DESC} : trie les resultats
par date de creation du plus recent au plus ancien.
41 \end{itemize}
42
43 \paragraph{Petit exemple illustratif}
44
45 Supposons ces petites tables :
46 \begin{lstlisting}[style=sql]
47 blogs:          users:          commentaires:
48 id | titre          id | nom          id | blog_id
49 1 | "Post A"        10 | "Alice"      100 | 1
50 2 | "Post B"        11 | "Bob"        101 | 1
51                102 | 2

```

Listing 11.3 – routes/blogs.js - Routes des blogs

Si on fait un LEFT JOIN de blogs avec commentaires, on obtient :

```

1 SELECT b.id, b.titre, c.id as commentaire_id
2 FROM blogs b
3 LEFT JOIN commentaires c ON b.id = c.blog_id;
4
5 -- Resultat:
6 -- 1 | "Post A" | 100
7 -- 1 | "Post A" | 101
8 -- 2 | "Post B" | 102

```

Appliquer COUNT(c.id) avec GROUP BY b.id donne le nombre de commentaires par blog :

```

1 SELECT b.id, b.titre, COUNT(c.id) as nb
2 FROM blogs b
3 LEFT JOIN commentaires c ON b.id = c.blog_id
4 GROUP BY b.id, b.titre;
5
6 -- Resultat:
7 -- 1 | "Post A" | 2
8 -- 2 | "Post B" | 1

```

Remarques pratiques :

- Toujours verifier les colonnes du GROUP BY : toutes les colonnes non-aggregees doivent y apparaitre.
- Utiliser LEFT JOIN si vous voulez garder les entites de gauche meme si elles n'ont pas de correspondance.

```

- Si vous utilisez JOIN (INNER JOIN), les lignes sans correspondance seront
  exclues.

// =====
// RECUPERER UN BLOG PAR ID // =====
router.get('/:id', async (req, res) => try const id = req.params;
  const result = await db.query( ' SELECT b.*, u.nom, u.prenom, u.email as
  auteur_email FROM blogs b LEFT JOIN users u ON b.auteur_id = u.id WHERE b.id = ' +
  [id]);
  if (result.rows.length === 0) return res.status(404).json( error: 'Blog
  non trouve' );
  res.json( blog: result.rows[0] ); catch (err) console.error('Erreur:',
  err); res.status(500).json( error: 'Erreur lors de la recuperation du blog'
  ); );
// =====
// CREER UN BLOG // =====
router.post('/', authMiddleware, checkRole('auteur', 'admin'), async (req,
  res) => try const titre, contenu = req.body; const auteur_id = req.user.userId;
  if (!titre || !contenu) return res.status(400).json( error: 'Titre et contenu
  obligatoires' );
  const result = await db.query( 'INSERT INTO blogs (titre, contenu, auteur_id) VALUES
  (2,3) RETURNING *', [titre, contenu, auteur_id]);
  res.status(201).json( message: 'Blog cree avec succes', blog: result.rows[0]
  ); catch (err) console.error('Erreur:', err); res.status(500).json( error:
  'Erreur lors de la creation du blog' ); );
// =====
// MODIFIER UN BLOG // =====
router.put('/:id', authMiddleware, checkRole('auteur', 'admin'), async (req,
  res) => try const id = req.params; const titre, contenu = req.body; const
  userId = req.user.userId; const userRole = req.user.role;
  // Verification que le blog existe const blog = await db.query('SELECT *
  FROM blogs WHERE id = ' + [id]);
  if (blog.rows.length === 0) return res.status(404).json( error: 'Blog non
  trouve' );
  // Verification des droits (auteur ou admin) if (userRole !== 'admin' blog.rows[0].
  userId) return res.status(403).json( error: 'Acces non autorise' );
  // Mise a jour du blog const result = await db.query( 'UPDATE blogs SET
  titre = 1, contenu = 2, date_modification = CURRENT_TIMESTAMP WHERE id = 3
  RETURNING *', [titre, contenu, id] );
  res.json( message: 'Blog modifie avec succes', blog: result.rows[0] ); catch
  (err) console.error('Erreur:', err); res.status(500).json( error: 'Erreur
  lors de la modification du blog' ); );
// =====
// SUPPRIMER UN BLOG // =====
router.delete('/:id', authMiddleware, checkRole('auteur', 'admin'), async
  (req, res) => try const id = req.params; const userId = req.user.userId;
  const userRole = req.user.role;

```

```

    // Verification que le blog existe const blog = await db.query('SELECT *
FROM blogs WHERE id = 1',[id]);
    if (blog.rows.length === 0) return res.status(404).json( error: 'Blog non
trouve' );
    // Verification des droits (auteur ou admin) if (userRole !== 'admin' blog.rows[0]
userId)returnres.status(403).json(error:'Accesnonautorise');
    // Suppression du blog await db.query('DELETE FROM blogs WHERE id = 1',[id]);
    res.json( message: 'Blog supprime avec succes' ); catch (err) console.error('Erreur
err); res.status(500).json( error: 'Erreur lors de la suppression du blog'
); );
    module.exports = router;

```

11.4 Routes des Commentaires

Creez le fichier routes/commentaires.js :

```

1 const express = require('express');
2 const db = require('../db');
3 const authMiddleware = require('../middleware/auth');
4
5 const router = express.Router();
6
7 //
8 // =====
9 // RECUPERER LES COMMENTAIRES D'UN BLOG
10 // =====
11
12 router.get('/blog/:blogId', async (req, res) => {
13   try {
14     const { blogId } = req.params;
15
16     const result = await db.query('
17       SELECT c.*, u.nom, u.prenom, u.email
18       FROM commentaires c
19       LEFT JOIN users u ON c.auteur_id = u.id
20       WHERE c.blog_id = $1
21       ORDER BY c.date_creation ASC
22     ', [blogId]);
23
24     res.json({ commentaires: result.rows });
25   } catch (err) {
26     console.error('Erreur:', err);
27     res.status(500).json({ error: 'Erreur lors de la recuperation
des commentaires' });
28   }
29 });
30 //
31 // =====

```

```

31 // CREER UN COMMENTAIRE
32 //
=====
33
34 router.post('/', authMiddleware, async (req, res) => {
35     try {
36         const { contenu, blog_id } = req.body;
37         const auteur_id = req.user.userId;
38
39         if (!contenu || !blog_id) {
40             return res.status(400).json({ error: 'Contenu et blog_id
obligatoires' });
41         }
42
43         // Verification que le blog existe
44         const blog = await db.query('SELECT id FROM blogs WHERE id = $1',
, [blog_id]);
45
46         if (blog.rows.length === 0) {
47             return res.status(404).json({ error: 'Blog non trouve' });
48         }
49
50         const result = await db.query(
51             'INSERT INTO commentaires (contenu, auteur_id, blog_id)
52             VALUES ($1, $2, $3)
53             RETURNING *',
54             [contenu, auteur_id, blog_id]
55         );
56
57         res.status(201).json({
58             message: 'Commentaire cree avec succes',
59             commentaire: result.rows[0]
60         });
61     } catch (err) {
62         console.error('Erreur:', err);
63         res.status(500).json({ error: 'Erreur lors de la creation du
commentaire' });
64     }
65 });
66
67 //
=====
68 // SUPPRIMER UN COMMENTAIRE
69 //
=====
70
71 router.delete('/:id', authMiddleware, async (req, res) => {
72     try {
73         const { id } = req.params;
74         const userId = req.user.userId;
75         const userRole = req.user.role;
76
77         // Verification que le commentaire existe

```

```
78     const commentaire = await db.query(  
79         'SELECT * FROM commentaires WHERE id = $1',  
80         [id]  
81     );  
82  
83     if (commentaire.rows.length === 0) {  
84         return res.status(404).json({ error: 'Commentaire non trouve  
85     }, {});  
86     }  
87  
88     // Verification des droits (auteur ou admin)  
89     if (userRole !== 'admin' && commentaire.rows[0].auteur_id !==  
90     userId) {  
91         return res.status(403).json({ error: 'Acces non autorise' })  
92     };  
93  
94     // Suppression du commentaire  
95     await db.query('DELETE FROM commentaires WHERE id = $1', [id]);  
96  
97     res.json({ message: 'Commentaire supprime avec succes' });  
98     } catch (err) {  
99         console.error('Erreur:', err);  
100         res.status(500).json({ error: 'Erreur lors de la suppression du  
101         commentaire' });  
102     }  
103 }  
104 });  
105  
106 module.exports = router;
```

Listing 11.4 – routes/commentaires.js - Routes des commentaires

11.5 Validation des Donnees

✓ Bonne Pratique

Regles de Validation :

- Toujours valider les donnees cote serveur
- Ne jamais faire confiance aux donnees du client
- Valider le type, le format et la longueur
- Retourner des messages d'erreur clairs
- Utiliser des codes HTTP appropries (400, 401, 403, 404)

11.6 Codes HTTP

Definition

Codes HTTP Courants :

- **200 OK** : Requete reussie
- **201 Created** : Ressource creee
- **400 Bad Request** : Requete invalide
- **401 Unauthorized** : Non authentifie
- **403 Forbidden** : Authentifie mais non autorise
- **404 Not Found** : Ressource non trouvee
- **500 Internal Server Error** : Erreur serveur

Chapitre 12

Prevention des Injections SQL

12.1 Qu'est-ce qu'une Injection SQL ?

Definition

Injection SQL : Vulnerabilite de securite qui permet a un attaquant d'injecter du code SQL malveillant dans une requete. Cela peut permettre de voler, modifier ou supprimer des donnees.

Erreur

Exemple d'Injection SQL :

Si vous construisez une requete par concatenation :

```
1 const email = req.body.email;
2 const query = 'SELECT * FROM users WHERE email = \'' + email + '\',
3 ;
```

Un attaquant peut envoyer :

```
1 email = "' OR '1'='1";
2
```

La requete devient :

```
1 SELECT * FROM users WHERE email = "' OR '1'='1';
```

Cette requete retourne TOUS les utilisateurs !

12.2 Prevention avec Parametres Positionnels

```
1 const email = req.body.email;
2 const result = await db.query(
3   'SELECT * FROM users WHERE email = $1',
4   [email]
5 );
```

Listing 12.1 – Requete securisee avec parametres

Definition

Parametres Positionnels : Placeholders (\$1, \$2, ...) dans les requetes SQL qui sont remplaces par les valeurs fournies. Le driver pg echappe automatiquement les valeurs, prevenant les injections SQL.

Bonne Pratique

Regle d'Or : **JAMAIS** concatener des chaines pour construire des requetes SQL. **TOUJOURS** utiliser des parametres positionnels.

12.3 Exemples de Requetes Securisees

```
1 // Requete avec un parametre
2 await db.query('SELECT * FROM users WHERE id = $1', [userId]);
3
4 // Requete avec plusieurs parametres
5 await db.query(
6     'INSERT INTO users (email, nom) VALUES ($1, $2)',
7     [email, nom]
8 );
9
10 // Requete avec parametre dans IN
11 await db.query(
12     'SELECT * FROM blogs WHERE id = ANY($1)',
13     [[id1, id2, id3]]
14 );
```

Listing 12.2 – Exemples de requetes securisees

Chapitre 13

Gestion des Erreurs

13.1 Types d'Erreurs

13.1.1 Erreurs Client (4xx)

- **400 Bad Request** : Requete invalide
- **401 Unauthorized** : Non authentifie
- **403 Forbidden** : Non autorise
- **404 Not Found** : Ressource non trouvee
- **409 Conflict** : Conflit (ex : email deja utilise)
- **422 Unprocessable Entity** : Donnees invalides

13.1.2 Erreurs Serveur (5xx)

- **500 Internal Server Error** : Erreur interne
- **503 Service Unavailable** : Service indisponible
- **504 Gateway Timeout** : Timeout

13.2 Try-Catch

```
1 router.post('/register', async (req, res) => {
2   try {
3     // Code qui peut echouer
4     const result = await db.query(...);
5     res.json({ user: result.rows[0] });
6   } catch (err) {
7     // Gestion de l'erreur
8     console.error('Erreur:', err);
9     res.status(500).json({ error: 'Erreur serveur' });
10  }
11 });
```

Listing 13.1 – Gestion des erreurs avec try-catch

13.3 Middleware d'Erreur Global

```
1 app.use((err, req, res, next) => {
2   console.error('Erreur:', err.stack);
3
4   // En production, ne pas exposer les details de l'erreur
5   if (process.env.NODE_ENV === 'production') {
6     res.status(500).json({ error: 'Erreur interne du serveur' });
7   } else {
8     res.status(500).json({
9       error: err.message,
10      stack: err.stack
11    });
12  }
13 });
```

Listing 13.2 – Middleware d'erreur global

✓ Bonne Pratique

Bonnes Pratiques :

- Toujours logger les erreurs cote serveur
- Ne jamais exposer les details de l'erreur au client en production
- Utiliser des codes HTTP appropries
- Fournir des messages d'erreur clairs mais non techniques
- Gerer les erreurs de base de donnees gracieusement

Chapitre 14

Frontend HTML/CSS/JS

14.1 Introduction au Frontend

Definition

Frontend : Partie de l'application visible par l'utilisateur. Elle est exécutée dans le navigateur et communique avec le backend via des requêtes HTTP.

14.2 Page d'Accueil (index.html)

Listing 14.1 – public/index.html - Page d'accueil

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0"
6         >
7     <title>Blog Multi-Role</title>
8     <link rel="stylesheet" href="css/style.css">
9 </head>
10 <body>
11     <header>
12         <h1>Blog Multi-Role</h1>
13         <nav>
14             <a href="index.html">Accueil</a>
15             <a href="login.html" id="loginLink">Connexion</a>
16             <a href="register.html" id="registerLink">Inscription</a>
17             <a href="#" id="logoutLink" style="display:none;">
18                 Deconnexion</a>
19         </nav>
20     </header>
21     <main>
22         <section id="blogsList">
23             <h2>Articles Recents</h2>
24             <div id="blogsContainer"></div>
```

```

24     </section>
25 </main>
26
27     <script src="js/app.js"></script>
28 </body>
29 </html>

```

14.3 Page de Connexion (login.html)

Listing 14.2 – public/login.html - Page de connexion

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Connexion - Blog Multi-Role</title>
7     <link rel="stylesheet" href="css/style.css">
8 </head>
9 <body>
10     <div class="container">
11         <h1>Connexion</h1>
12         <form id="loginForm">
13             <div>
14                 <label for="email">Email :</label>
15                 <input type="email" id="email" name="email" required>
16             </div>
17             <div>
18                 <label for="password">Mot de passe :</label>
19                 <input type="password" id="password" name="password"
20                     required>
21             </div>
22             <button type="submit">Se connecter</button>
23         </form>
24         <p>Pas encore inscrit ? <a href="register.html">S'inscrire</a></p>
25         <div id="errorMessage"></div>
26     </div>
27     <script src="js/login.js"></script>
28 </body>
29 </html>

```

14.4 JavaScript Frontend (app.js)

```

1 const API_URL = 'http://localhost:3000/api';
2
3 function getAuthHeaders() {
4     const accessToken = localStorage.getItem('accessToken');

```



```
5     return accessToken ? { Authorization: 'Bearer ${accessToken}' } :
6     {};
7 }
8 async function refreshAccessToken() {
9     const refreshToken = localStorage.getItem('refreshToken');
10    if (!refreshToken) {
11        return null;
12    }
13
14    const response = await fetch(`${API_URL}/auth/token`, {
15        method: 'POST',
16        headers: {
17            'Content-Type': 'application/json'
18        },
19        body: JSON.stringify({ refreshToken })
20    });
21
22    if (!response.ok) {
23        return null;
24    }
25
26    const data = await response.json();
27    localStorage.setItem('accessToken', data.accessToken);
28    return data.accessToken;
29 }
30
31 // Verification de l'authentification au chargement
32 document.addEventListener('DOMContentLoaded', () => {
33     const accessToken = localStorage.getItem('accessToken');
34
35     if (accessToken) {
36         document.getElementById('loginLink').style.display = 'none';
37         document.getElementById('registerLink').style.display = 'none';
38         document.getElementById('logoutLink').style.display = 'block';
39     }
40
41     loadBlogs();
42 });
43
44 // Chargement des blogs
45 async function loadBlogs() {
46     try {
47         let response = await fetch(`${API_URL}/blogs`, {
48             headers: getAuthHeaders()
49         });
50
51         if (response.status === 401) {
52             const newAccessToken = await refreshAccessToken();
53             if (newAccessToken) {
54                 response = await fetch(`${API_URL}/blogs`, {
55                     headers: { Authorization: 'Bearer ${newAccessToken}'
56
57                 });
58             }
59         }
60     }
61 }
```

```

59
60     const data = await response.json();
61
62     const container = document.getElementById('blogsContainer');
63     container.innerHTML = '';
64
65     data.blogs.forEach(blog => {
66         const blogElement = document.createElement('article');
67         blogElement.className = 'blog-card';
68         blogElement.innerHTML = `
69             <h3>${blog.titre}</h3>
70             <p class="author">Par ${blog.nom} ${blog.prenom}</p>
71             <p class="date">${new Date(blog.date_creation).
toLocaleDateString('fr-FR')}</p>
72             <p>${blog.contenu.substring(0, 200)}...</p>
73             <a href="blog.html?id=${blog.id}">Lire la suite</a>
74         `;
75         container.appendChild(blogElement);
76     });
77 } catch (error) {
78     console.error('Erreur:', error);
79 }
80 }
81
82 // Deconnexion
83 document.getElementById('logoutLink').addEventListener('click', (e) => {
84     e.preventDefault();
85     localStorage.removeItem('accessToken');
86     localStorage.removeItem('refreshToken');
87     localStorage.removeItem('user');
88     window.location.href = 'login.html';
89 });

```

Listing 14.3 – public/js/app.js - JavaScript principal

14.5 JavaScript de Connexion (login.js)

```

1  const API_URL = 'http://localhost:3000/api';
2
3  document.getElementById('loginForm').addEventListener('submit', async (e) => {
4      e.preventDefault();
5
6      const email = document.getElementById('email').value;
7      const password = document.getElementById('password').value;
8
9      try {
10         const response = await fetch(`${API_URL}/auth/login`, {
11             method: 'POST',
12             headers: {
13                 'Content-Type': 'application/json'
14             },
15             body: JSON.stringify({ email, password })
16         });

```

```
17
18     const data = await response.json();
19
20     if (response.ok) {
21         localStorage.setItem('accessToken', data.accessToken);
22         localStorage.setItem('refreshToken', data.refreshToken);
23         localStorage.setItem('user', JSON.stringify(data.user));
24         window.location.href = 'index.html';
25     } else {
26         document.getElementById('errorMessage').textContent = data.
error;
27     }
28 } catch (error) {
29     console.error('Erreur:', error);
30     document.getElementById('errorMessage').textContent = 'Erreur de
connexion';
31 }
32 });
```

Listing 14.4 – public/js/login.js - JavaScript de connexion

14.6 CSS (style.css)

Listing 14.5 – public/css/style.css - Styles CSS

```
1 * {
2     margin: 0;
3     padding: 0;
4     box-sizing: border-box;
5 }
6
7 body {
8     font-family: Arial, sans-serif;
9     line-height: 1.6;
10    background-color: #f4f4f4;
11 }
12
13 header {
14     background-color: #333;
15     color: white;
16     padding: 1rem;
17 }
18
19 header nav a {
20     color: white;
21     text-decoration: none;
22     margin-right: 1rem;
23 }
24
25 .container {
26     max-width: 800px;
27     margin: 2rem auto;
28     padding: 2rem;
29     background: white;
```

```
30     border-radius: 5px;
31 }
32
33 .blog-card {
34     background: white;
35     padding: 1.5rem;
36     margin-bottom: 1rem;
37     border-radius: 5px;
38     box-shadow: 0 2px 5px rgba(0,0,0,0.1);
39 }
40
41 form div {
42     margin-bottom: 1rem;
43 }
44
45 form label {
46     display: block;
47     margin-bottom: 0.5rem;
48 }
49
50 form input {
51     width: 100%;
52     padding: 0.5rem;
53     border: 1px solid #ddd;
54     border-radius: 3px;
55 }
56
57 button {
58     background-color: #333;
59     color: white;
60     padding: 0.5rem 1rem;
61     border: none;
62     border-radius: 3px;
63     cursor: pointer;
64 }
65
66 button:hover {
67     background-color: #555;
68 }
```

Chapitre 15

Debogage et Resolution de Problemes

15.1 Introduction au Debogage

Definition

Debogage (Debugging) : Processus systematique d'identification, d'analyse et de resolution des erreurs (bugs) dans un logiciel.

15.2 Outils de Debogage

15.2.1 console.log()

```
1 console.log('Variable:', variable);  
2 console.log('Requete:', req.body);  
3 console.log('Resultat:', result.rows);
```

Listing 15.1 – Debogage avec console.log

Bonne Pratique

Bonnes Pratiques de Logging :

- Logger des informations contextuelles (pas juste des valeurs)
- Utiliser des niveaux de log (info, warn, error)
- Ne pas logger d'informations sensibles (mots de passe, tokens)
- Supprimer les logs de debug en production

15.2.2 Node.js Inspector

Listing 15.2 – Demarrage avec le debugger

```
1 node --inspect server.js
```

Definition

Node.js Inspector : Interface de debugage integree a Node.js qui permet de deboguer le code avec Chrome DevTools ou VS Code.

15.2.3 VS Code Debugger

Listing 15.3 – .vscode/launch.json - Configuration VS Code

```
1 {  
2   "version": "0.2.0",  
3   "configurations": [  
4     {  
5       "type": "node",  
6       "request": "launch",  
7       "name": "Lancer le serveur",  
8       "program": "${workspaceFolder}/server.js"  
9     }  
10  ]  
11 }
```

15.3 Erreurs Courantes Node.js

15.3.1 Module Not Found

Erreur

Erreur : Cannot find module 'express'

Cause : La dependance n'est pas installee.

Resolution :

```
1 npm install
```

15.3.2 EADDRINUSE

! Erreur

Erreur : EADDRINUSE : address already in use : : 3000

Cause : Le port 3000 est déjà utilisé par un autre processus.

Resolution :

```
1 # Trouver le processus
2 netstat -ano | findstr :3000
3
4 # Tuer le processus (Windows)
5 taskkill /PID <PID> /F
6
7 # Ou utiliser un autre port
8 PORT=3001 npm start
```

15.4 Erreurs Courantes PostgreSQL

15.4.1 Connection Refused

! Erreur

Erreur : connection refused

Cause : PostgreSQL n'est pas démarré.

Resolution :

```
1 # Démarrer PostgreSQL
2 sudo service postgresql start
```

15.4.2 Password Authentication Failed

! Erreur

Erreur : password authentication failed

Cause : Mot de passe incorrect.

Resolution : Vérifiez le mot de passe dans `.env`.

15.5 Strategie de Debogage

1. Reproduire l'erreur de manière fiable
2. Isoler le problème (éliminer les facteurs externes)
3. Ajouter des logs pour comprendre le flux
4. Hypothèse sur la cause

-
5. Tester l'hypothese
 6. Corriger et verifier

Chapitre 16

Deploiement et Production

16.1 Introduction au Deploiement

Definition

Deploiement : Processus de mise en production d'une application, c'est-à-dire de la rendre accessible aux utilisateurs finaux sur un serveur.

16.2 Preparation pour la Production

16.2.1 Variables d'Environnement

Attention

Securite Critique :

- Utilisez des valeurs differentes en production
- Changez le JWT_SECRET
- Changez les mots de passe
- Utilisez un gestionnaire de secrets (AWS Secrets Manager, HashiCorp Vault)

16.2.2 NODE_ENV

```
1 export NODE_ENV=production
```

Definition

NODE_ENV : Variable d'environnement qui indique a Node.js dans quel environnement il s'execute. En production, cela active certaines optimisations.

16.3 PM2 - Process Manager

Definition

PM2 : Gestionnaire de processus pour Node.js qui permet de garder les applications en vie, de les redemarrer automatiquement en cas de crash, et de gerer le clustering.

16.3.1 Installation de PM2

```
1 npm install -g pm2
```

16.3.2 Demarrage avec PM2

Listing 16.1 – Demarrage de l'application avec PM2

```
1 pm2 start server.js --name "blog-api"
2
3 pm2 save
4
5 pm2 startup
```

16.3.3 Commandes PM2

```
1 pm2 list           # Lister les processus
2 pm2 logs           # Voir les logs
3 pm2 stop blog-api  # Arrêter
4 pm2 restart blog-api # Redemarrer
5 pm2 delete blog-api # Supprimer
6 pm2 monit          # Moniteur en temps reel
```

16.4 Nginx - Reverse Proxy

Definition

Reverse Proxy : Serveur qui recoit les requetes des clients et les transmet aux serveurs backend. Il fournit une couche d'abstraction et de securite.

16.4.1 Configuration Nginx

Listing 16.2 – /etc/nginx/sites-available/blog

```
1 server {
2     listen 80;
3     server_name votre-domaine.com;
4 }
```

```
5     location / {
6         proxy_pass http://localhost:3000;
7         proxy_http_version 1.1;
8         proxy_set_header Upgrade $http_upgrade;
9         proxy_set_header Connection 'upgrade';
10        proxy_set_header Host $host;
11        proxy_cache_bypass $http_upgrade;
12        proxy_set_header X-Real-IP $remote_addr;
13        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
14        proxy_set_header X-Forwarded-Proto $scheme;
15    }
16 }
```

16.5 HTTPS avec Let's Encrypt

```
1 # Installation de Certbot
2 sudo apt-get install certbot python3-certbot-nginx
3
4 # Obtention du certificat
5 sudo certbot --nginx -d votre-domaine.com
6
7 # Renouvellement automatique
8 sudo certbot renew --dry-run
```

16.6 Base de Donnees en Production

✓ Bonne Pratique

Bonnes Pratiques :

- Utilisez une base de donnees geree (AWS RDS, Google Cloud SQL)
- Configurez les backups automatiques
- Utilisez des replicas pour la haute disponibilite
- Configurez le monitoring
- Limitez l'accès par IP

Chapitre 17

Bonnes Pratiques de Developpement

17.1 Clean Code

Definition

Clean Code : Code qui est facile a comprendre, a modifier et a maintenir. Il suit des principes de clarte, de simplicité et d'organisation.

17.1.1 Noms Descriptifs

```
1 // Mauvais
2 const d = new Date();
3 const u = await getUser(id);
4
5 // Bon
6 const currentDate = new Date();
7 const user = await getUser(userId);
```

Listing 17.1 – Mauvais vs Bon nommage

17.1.2 Fonctions Courtes

Bonne Pratique

Regle : Une fonction ne devrait faire qu'une seule chose et ne devrait pas dépasser 20-30 lignes.

17.2 Principe DRY

Definition

DRY (Don't Repeat Yourself) : Principe qui stipule que chaque piece de connaissance doit avoir une representation unique et non ambiguë dans le systeme.

```
1 // Mauvais
2 router.get('/users', async (req, res) => {
3   try {
4     const result = await db.query('SELECT * FROM users');
5     res.json(result.rows);
6   } catch (err) {
7     console.error(err);
8     res.status(500).json({ error: 'Erreur serveur' });
9   }
10 });
11
12 router.get('/blogs', async (req, res) => {
13   try {
14     const result = await db.query('SELECT * FROM blogs');
15     res.json(result.rows);
16   } catch (err) {
17     console.error(err);
18     res.status(500).json({ error: 'Erreur serveur' });
19   }
20 });
21
22 // Bon - Utiliser un middleware de gestion d'erreurs
```

Listing 17.2 – Eviter la repetition

17.3 Principes SOLID

Definition

SOLID : Acronyme pour cinq principes de conception orientee objet :

- **S** : Single Responsibility Principle (une seule responsabilite)
- **O** : Open/Closed Principle (ouvert a l'extension, ferme a la modification)
- **L** : Liskov Substitution Principle (substitution de Liskov)
- **I** : Interface Segregation Principle (segregation des interfaces)
- **D** : Dependency Inversion Principle (inversion des dependances)

17.4 Validation des Donnees

✓ Bonne Pratique

Regles de Validation :

- Valider cote serveur (jamais faire confiance au client)
- Valider le type, le format et la longueur
- Sanitizer les entrees (echapement XSS)
- Utiliser des bibliotheques de validation (Joi, Zod)

17.5 Gestion des Secrets

Securite

Regles d'Or :

- JAMAIS hardcoder les secrets
- Utiliser des variables d'environnement
- Ne jamais committer les fichiers `.env`
- Utiliser des gestionnaires de secrets en production
- Faire la rotation des secrets regulierement

17.6 Documentation

✓ Bonne Pratique

Documentation Essentielle :

- README.md avec instructions d'installation
- Commentaires pour le code complexe
- Documentation des API (Swagger/OpenAPI)
- Changelog pour les changements

Chapitre 18

Extensions et Ameliorations Possibles

18.1 Introduction

Ce chapitre presente des extensions et ameliorations possibles pour rendre le blog plus complet et professionnel.

18.2 Systeme de Likes

Listing 18.1 – Table des likes

```
1 CREATE TABLE likes (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     user_id UUID NOT NULL,  
4     blog_id UUID NOT NULL,  
5     date_creation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
6     UNIQUE(user_id, blog_id),  
7     CONSTRAINT fk_user_like FOREIGN KEY (user_id) REFERENCES users(id)  
8         ON DELETE CASCADE,  
9     CONSTRAINT fk_blog_like FOREIGN KEY (blog_id) REFERENCES blogs(id)  
10        ON DELETE CASCADE  
11 );
```

18.3 Tags et Categories

Le systeme de tags est deja implemente dans le schema de base de donnees. Il peut etre etendu avec :

- Filtrage par tag
- Suggestions de tags automatiques
- Nuage de tags

18.4 Recherche Full-Text

Listing 18.2 – Index full-text pour la recherche

```
1 CREATE INDEX idx_blogs_search ON blogs
2 USING gin(to_tsvector('french', titre || ' ' || contenu));
```

18.5 Pagination

```
1 router.get('/', async (req, res) => {
2   const page = parseInt(req.query.page) || 1;
3   const limit = parseInt(req.query.limit) || 10;
4   const offset = (page - 1) * limit;
5
6   const result = await db.query(
7     'SELECT * FROM blogs ORDER BY date_creation DESC LIMIT $1 OFFSET
8     $2',
9     [limit, offset]
10  );
11  res.json({ blogs: result.rows, page, limit });
12 });
```

Listing 18.3 – Implementation de la pagination

18.6 Upload d'Images

Definition

Upload d'Images : Fonctionnalité permettant aux utilisateurs d'ajouter des images à leurs blogs. Nécessite un stockage d'objets (AWS S3, Cloudinary) ou un stockage local.

18.7 Notifications par Email

Definition

Notifications Email : Envoi automatique d'emails pour les nouveaux commentaires, les likes, etc. Utilise des services comme SendGrid, Mailgun, ou AWS SES.

18.8 Refresh Tokens

Definition

Refresh Token : Jeton à longue durée de vie, conserve côté client et côté serveur, utilise pour obtenir un nouveau access token sans forcer l'utilisateur à se reconnecter.

18.8.1 Pourquoi utiliser un refresh token ?

- Permet de garder une expiration courte sur l'access token (plus securise)
- Evite de renvoyer les identifiants de l'utilisateur a chaque requete
- Limite les dommages si un access token est vole : il n'est valide que quelques minutes
- Rend possible la deconnexion centralisee en invalidant le refresh token cote serveur

18.8.2 Flux typique

1. L'utilisateur se connecte avec email/mot de passe
2. Le serveur renvoie un `accessToken` court terme et un `refreshToken` long terme
3. Le client utilise l'`accessToken` dans l'en-tete `Authorization: Bearer ...` pour les routes proteges
4. Si l'`accessToken` expire, le client envoie le `refreshToken` au serveur via `POST /auth/token`
5. Le serveur verifie le refresh token et renvoie un nouveau `accessToken`

```
1 async function refreshAccessToken() {
2   const refreshToken = localStorage.getItem('refreshToken');
3   const response = await fetch(`${API_URL}/auth/token`, {
4     method: 'POST',
5     headers: {
6       'Content-Type': 'application/json'
7     },
8     body: JSON.stringify({ refreshToken })
9   });
10
11   if (!response.ok) {
12     return null;
13   }
14
15   const data = await response.json();
16   localStorage.setItem('accessToken', data.accessToken);
17   return data.accessToken;
18 }
```

Listing 18.4 – Vue front-end du rafraichissement du token

18.9 Rate Limiting

```
1 const rateLimit = require('express-rate-limit');
2
3 const loginLimiter = rateLimit({
4   windowMs: 15 * 60 * 1000, // 15 minutes
5   max: 5, // 5 tentatives
6   message: 'Trop de tentatives, reessayez plus tard'
7 });
8
9 router.post('/login', loginLimiter, async (req, res) => {
```

```
10 // ...  
11 });
```

Listing 18.5 – Implementation du rate limiting

18.10 WebSocket pour Commentaires en Temps Reel

Definition

WebSocket : Protocole de communication bidirectionnelle qui permet d'envoyer des messages en temps reel entre le client et le serveur.

18.11 Caching avec Redis

Definition

Redis : Base de donnees en memoire utilisee pour le caching, les sessions, et les files d'attente. Peut considerablement ameliorer les performances.

Chapitre 19

Conclusion

19.1 Recapitulatif du Projet

Ce projet pedagogique nous a permis de developper une application web complete de blog multi-role en utilisant les technologies modernes suivantes :

- **Backend** : Node.js avec Express.js
- **Base de donnees** : PostgreSQL avec le driver pg
- **Authentification** : JWT (JSON Web Tokens)
- **Securite** : bcrypt pour le hachage des mots de passe
- **Frontend** : HTML, CSS et JavaScript vanilla

19.2 Competences Acquises

19.2.1 Competences Techniques

- Configuration d'un serveur Express.js
- Conception et implementation d'une base de donnees relationnelle
- Utilisation du driver PostgreSQL pg
- Implementation de l'authentification JWT
- Securite des mots de passe avec bcrypt
- Prevention des injections SQL
- Gestion des erreurs et des middlewares
- Developpement frontend avec HTML/CSS/JS

19.2.2 Competences en Securite

- Hachage des mots de passe
- Authentification stateless avec JWT
- Controle d'accès basé sur les rôles (RBAC)
- Prevention des injections SQL
- Gestion securisee des secrets

19.2.3 Competences en Architecture

- Architecture client-serveur
- Separation des preoccupations (middleware, routes)
- Modelisation relationnelle
- API RESTful

19.3 Perspectives d'Avenir

19.3.1 Poursuite de l'Apprentissage

Pour continuer a progresser, vous pouvez explorer :

- **Frameworks Frontend** : React, Vue.js, Angular
- **ORM** : Sequelize, TypeORM, Prisma
- **Tests** : Jest, Mocha, Chai
- **CI/CD** : GitHub Actions, GitLab CI
- **Conteneurisation** : Docker, Kubernetes
- **Cloud** : AWS, Google Cloud, Azure

19.3.2 Ameliorations du Projet

- Ajouter des tests unitaires et d'integration
- Implementer le systeme de likes
- Ajouter la recherche full-text
- Implementer le caching avec Redis
- Ajouter des notifications par email
- Creer une interface d'administration
- Ajouter des statistiques et analytics

19.4 Conclusion Finale

Ce projet a ete une excellente introduction au developpement backend moderne. Il a permis de comprendre les concepts fondamentaux du developpement web, de la securite, et de l'architecture des applications.

Les competences acquises constituent une base solide pour poursuivre dans le developpement d'applications web plus complexes et professionnelles.

B

ibliographie

Documentation Officielle

- Node.js Documentation. <https://nodejs.org/docs/>
- Express.js Documentation. <https://expressjs.com/>
- PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- pg (Node.js PostgreSQL Driver). <https://node-postgres.com/>
- JWT (JSON Web Tokens). <https://jwt.io/>
- bcrypt Documentation. <https://github.com/kelektiv/node.bcrypt.js>

Ressources en Ligne

- MDN Web Docs. <https://developer.mozilla.org/>
- OWASP Top 10. <https://owasp.org/www-project-top-ten/>
- REST API Tutorial. <https://restfulapi.net/>

Livres Recommandes

- *Node.js Design Patterns* par Mario Casciaro
- *You Don't Know JS* par Kyle Simpson
- *Clean Code* par Robert C. Martin